

A Chi Ha Sempre Creduto In Me.



**UNIVERSITÀ
DI TORINO**

Università degli Studi di Torino

Corso di Laurea in Informatica

OpenDSA: Reading

Tesi di Laurea

Relatore:

Esposito Roberto

Candidato:

De Marco Lorenzo

Matricola 920033

Anno Accademico 2024/2025

Indice

1	Introduzione	1
1.1	La Dislessia	1
1.1.1	Definizione e criteri diagnostici	1
1.1.2	Prevalenza ed epidemiologia	2
1.1.3	Cause e fattori neurobiologici	3
1.1.4	Caratteristiche cliniche e sintomi principali . . .	3
1.1.5	Diagnosi e valutazione	4
1.1.6	Interventi e strategie di supporto	5
1.1.7	Aspetti emotivi e psicologici	6
1.1.8	Vita adulta e prospettive future	7
1.1.9	In Breve	8
1.2	L'Avvento delle Nuove Tecnologie	9
1.2.1	Il concetto di accessibilità e l'impatto delle tecnologie	9
1.2.2	Le WCAG 2.0: linee guida per il web	10
1.2.3	Origine e struttura delle WCAG:	11
1.2.4	I quattro principi fondamentali (POUR)	11
1.2.5	Adozione e benefici	13
1.2.6	Sviluppo di software e hardware "user friendly"	13
1.2.7	Sviluppo software inclusivo	13
1.2.8	Linee guida e toolkit	14
1.2.9	Rispetto degli standard	14

1.2.10	Sviluppo hardware inclusivo	14
1.2.11	Esempi pratici di tecnologie e progetti inclusivi	15
1.2.12	Benefici secondari dell'accessibilità	16
1.2.13	Legislazione e normative di riferimento	17
1.2.14	Prospettive future	18
1.3	L'Approccio alla Dislessia con le Nuove Tecnologie . . .	20
1.3.1	L'Informatica ed i Moderni Approcci Tecnologici	20
1.3.2	Prospettive future	22
1.4	Il Ruolo della IA Generativa nel campo DSA	23
1.4.1	Cosa si intende per Intelligenza Artificiale generativa	23
1.4.2	Le radici storiche dell'IA: dagli albori ai primi modelli generativi	23
1.4.3	L'avvento dei sistemi conversazionali (anni '60-'70)	24
1.4.4	Il salto di qualità: reti neurali e apprendimento profondo	24
1.4.5	L'era del Deep Learning (anni 2000-2010) . . .	25
1.4.6	La svolta generativa: dalle GAN ai Transformer	25
1.4.7	La rivoluzione dei Transformer	26
1.4.8	L'impatto culturale della Generative AI	26
1.4.9	Rivoluzione sociale ed economica	27
1.4.10	Perché è stata (ed è) rivoluzionaria	28
1.4.11	In Breve	28
2	Tecnologie Utilizzate	31
2.1	Il Framework Flutter	31
2.2	Cenni storici	32
2.2.1	Il progetto Sky: i prodromi di Flutter	32
2.2.2	L'annuncio ufficiale di Flutter	32
2.2.3	Versione stabile e adozione in crescita	33
2.3	Perché Flutter è stato rivoluzionario?	33
2.3.1	Unico framework per più piattaforme	33

2.3.2	Hot Reload e sviluppo rapido	34
2.3.3	Widget personalizzabili e design coerente	34
2.4	La Rivoluzione di Flutter	34
2.4.1	Unificazione dello sviluppo cross-platform	35
2.4.2	Prestazioni elevate	35
2.4.3	Community e ecosistema in rapida espansione	35
2.4.4	Hot Reload e Produttività	36
2.5	Evoluzione e impatto culturale/industriale	37
2.5.1	Adozione da parte di aziende e sviluppatori	37
2.5.2	Convergenza tra mobile, desktop e web	37
2.5.3	Ruolo didattico e formativo	37
2.6	In Definitiva	38
2.7	Dart: un Linguaggio Multipiattaforma	39
2.7.1	La necessità di un nuovo linguaggio per il web	39
2.7.2	Il lancio di Dart	39
2.8	Principi fondamentali e prime versioni	41
2.8.1	Flessibilità e portabilità	41
2.8.2	Dart 1.0 (2013)	41
2.9	Evoluzione e caratteristiche rivoluzionarie	42
2.9.1	Dall'interpretazione alla compilazione AOT	42
2.9.2	Dart 2.0 (2018) e la tipizzazione forte	42
2.9.3	Dart DevTools e Hot Reload	42
2.10	Dart e Flutter: un legame strategico	43
2.10.1	La Simbiosi	43
2.10.2	I vantaggi di Dart nel contesto mobile	43
2.11	Come Dart è stato rivoluzionario	44
2.12	Adozione industriale e prospettive	45
2.13	In Conclusione	45
2.14	Il Modello VOSK	46
2.14.1	Origini e contesto storico: L'Evoluzione dei sistemi di riconoscimento vocale	46
2.14.2	Kaldi e la nascita di VOSK	46

2.15	Caratteristiche tecniche di VOSK	47
2.15.1	Funzionamento offline	47
2.15.2	Supporto multilingua e modelli pre-addestrati	47
2.15.3	Architettura modulare	48
2.15.4	Licenza Open Source	48
2.16	L'approccio open source: vantaggi rispetto alle soluzioni closed	48
2.17	L'integrazione di VOSK in Flutter	49
2.17.1	Plugin e pacchetti di terze parti	49
2.17.2	Processo di integrazione nel Progetto	49
2.17.3	Vantaggi di VOSK su Flutter	50
2.18	Perché VOSK è rivoluzionario	51
2.19	Riflessioni sul futuro e prospettive	51
2.20	In Conclusione	52
3	Processo di Sviluppo del Software	53
3.1	La Scelta del Cross-Platform	53
3.2	Design del Software	56
3.2.1	Alberatura del Progetto	56
3.2.2	Struttura del Progetto	57
3.2.3	Origini del concetto di architettura a livelli	57
3.2.4	Principi fondanti	58
3.3	Struttura tipica di un progetto Flutter	58
3.3.1	Il contesto Flutter	58
3.4	Descrizione dei livelli	59
3.5	Vantaggi specifici dell'approccio	61
3.5.1	Manutenibilità e testabilità	61
3.5.2	Scalabilità	62
3.5.3	Principio di "Single Responsibility"	62
3.6	Criticità e limiti	62
3.7	Pattern collegati in Flutter	63
3.7.1	BLoC (Business Logic Component)	63

3.7.2	Provider / Riverpod	63
3.7.3	Clean Architecture	63
3.7.4	Uso dell'Architettura a Livelli	63
3.8	L'Uso della Gamification	64
3.8.1	Origini e Definizione	64
3.8.2	Meccaniche e Dinamiche di Gioco	64
3.8.3	Design Pattern Usati	65
4	Sviluppo	67
4.1	Le Meccaniche di Gioco nella User Journey	67
4.1.1	I Trofei	68
4.1.2	La Durata di Gioco	68
4.1.3	Gli Esercizi	69
4.1.4	Il Profilo	69
4.2	L'Interfaccia Grafica	71
4.2.1	La Schermata di Avvio	71
4.2.2	La Selezione Del Profilo	72
4.2.3	La Schermata di Gioco Principale	75
4.2.4	Le Sfide	77
4.2.5	I Trofei	78
4.2.6	Gli Esercizi	79
4.3	Algoritmo di Levenshtein	79
4.3.1	Uso e applicazioni	80
4.3.2	Proprietà e varianti	81
4.3.3	Formalizzazione della distanza	81
4.3.4	Descrizione dell'algoritmo	82
4.3.5	Esempio di calcolo dettagliato	83
4.3.6	Analisi dell'algoritmo	84
4.3.7	Complessità	84
4.3.8	Ottimizzazioni	84
4.3.9	Il Feedback dell'Esercizio	88
4.4	Il Profilo Admin	92

4.5	Possibili Implementazioni Future	93
5	Problemi riscontrati e Soluzioni Proposte	95
5.1	Creare gli Eseguibili con Flutter	95
5.2	Apple e le sue Politiche di Distribuzione del Software .	96
5.3	La Scelta di Recorder come Plugin di Registrazione . .	97
5.4	I Problemi di un Ambiente non Trattato per l'Audio .	98
6	Conclusioni	99
7	Appendice A - Elementi di Gioco	101
7.1	Sfide	101
7.2	Trofei	103
7.3	Livelli	106
7.4	Configurazioni dell'Applicazione	108
8	Appendice B - Codice	113
8.1	VoskService	113
8.2	SpeechRecognitionService	126
8.3	AudioService	134

Capitolo 1

Introduzione

1.1 La Dislessia

1.1.1 Definizione e criteri diagnostici

La dislessia, termine che deriva dal greco "dys" (indicante una condizione di inadeguatezza) e "lexis" (riferito al linguaggio), rappresenta una condizione neurologica specifica che influenza l'apprendimento della lettura. Indica una difficoltà specifica nell'acquisizione delle abilità di lettura (correttezza e rapidità) che non è attribuibile a un deficit cognitivo generale, a deficit sensoriali o a condizioni di svantaggio socio-culturale.

Secondo il DSM-5 [3], la dislessia rientra sotto la categoria dei "Disturbi Specifici dell'Apprendimento" con compromissione della lettura. Gli studi di [stella2011] e Come evidenziato da [6] hanno contribuito significativamente alla comprensione di questo disturbo nel contesto italiano.. Vengono individuati criteri clinici specifici, quali:

- Persistenti difficoltà nella lettura di parole singole,
- Accuratezza e/o velocità nella lettura sostanzialmente al di sotto di quanto atteso per l'età cronologica,
- Esordio delle difficoltà in età scolare,
- Significativo impatto sul rendimento scolastico o sulle attività quotidiane che richiedono la lettura.

[3], [ida2024]

1.1.2 Prevalenza ed epidemiologia

Un aspetto importante evidenziato da [11] è che molte persone con dislessia sviluppano capacità cognitive uniche, come una spiccata creatività visuo-spaziale e un'ottima capacità di problem solving, che possono rappresentare vantaggi significativi in determinati contesti professionali e creativi. Le stime di prevalenza della dislessia variano a seconda delle definizioni adottate e dei criteri diagnostici. In generale, si stima che la dislessia interessi dal 3% al 10% della popolazione scolastica (stima ipotetica). In Italia, secondo i dati più recenti del MIUR [19], si è registrato un incremento significativo delle diagnosi, grazie all'implementazione della Legge 170/2010 [20] che ha fornito un quadro normativo chiaro per la gestione dei DSA nelle scuole.

Possibili motivazioni dell'incremento delle diagnosi:

- Maggior diffusione di conoscenze sulla dislessia.
- Miglioramento delle procedure di valutazione.
- Applicazione di normative e linee guida nazionali, come la Legge 170/2010, che tutela gli studenti con DSA.

1.1.3 Cause e fattori neurobiologici

Le basi neurobiologiche della dislessia sono supportate da numerose ricerche in ambito neuroscientifico. Come evidenziato da [6] e [26], gli studi di neuroimaging hanno mostrato (fMRI, PET, risonanza magnetica strutturale) hanno evidenziato differenze funzionali in alcune aree cerebrali coinvolte nella lettura e nell'elaborazione fonologica, in particolare:

Area temporo-parietale: essenziale per la corrispondenza suono-simbolo (correlazione tra fonemi e grafemi).

Area occipito-temporale: deputata al riconoscimento rapido delle parole scritte.

Area frontale inferiore (area di Broca): coinvolta nella produzione del linguaggio e nell'elaborazione fonologica.

Fattori genetici giocano un ruolo importante: la dislessia ha una tendenza familiare, e diversi geni sono stati associati all'insorgenza del disturbo (ad es. DCDC2, KIAA0319). Ciononostante, è fondamentale sottolineare che i meccanismi genetici non spiegano tutte le variabili e l'ambiente (metodologie didattiche, stimoli culturali) può modulare in parte l'espressione del disturbo.

1.1.4 Caratteristiche cliniche e sintomi principali

Le difficoltà più comuni riguardano:

Lettura lenta o scorretta: il bambino o la bambina spesso fa errori di sostituzione, omissione o inversione di lettere.

Difficoltà di decodifica fonologica: problemi nell'associare i suoni (fonemi) alle lettere scritte (grafemi).

Difficoltà nella comprensione del testo: la fatica nell'ottenere un'adeguata velocità di lettura influisce sulla comprensione dei contenuti.

Errori ortografici: sebbene non sempre compaiano nel quadro dislessico puro, possono presentarsi frequenti errori di ortografia legati alla scarsa automatizzazione.

Disfluenza nella lettura ad alta voce: il bambino può leggere in modo frammentato e monotono.

Oltre alle difficoltà di lettura, possono manifestarsi ulteriori problematiche associate, come bassa autostima, scarsa motivazione scolastica e possibili disagi emotivi legati all'esperienza di insuccesso ripetuto.

1.1.5 Diagnosi e valutazione

Un aspetto importante evidenziato da [11] è che molte persone con dislessia sviluppano capacità cognitive uniche, come una spiccata creatività visuo-spaziale e un'ottima capacità di problem solving, che possono rappresentare vantaggi significativi in determinati contesti professionali e creativi. La diagnosi di dislessia viene solitamente effettuata da un team multidisciplinare composto da neuropsichiatri infantili, psicologi e logopedisti. In Italia, i protocolli valutativi prevedono:

- Colloquio clinico con i genitori e con il bambino/ragazzo per raccogliere informazioni su sviluppo, storia medica e scolastica.
- Test neuropsicologici e prove standardizzate di lettura, per valutare la velocità e l'accuratezza.
- Valutazione del quoziente intellettivo (QI) e dello sviluppo cognitivo generale, per escludere disturbi intellettivi.
- Screening visivo e uditivo per escludere eventuali deficit sensoriali.

- Osservazione del contesto scolastico (quando possibile) per individuare gli ostacoli reali incontrati nell'apprendimento.

Un aspetto importante è la diagnosi differenziale rispetto ad altri disturbi dell'apprendimento (disortografia, disgrafia, discalculia) o a possibili comorbidità (es. disturbo da deficit di attenzione/iperattività – ADHD).

1.1.6 Interventi e strategie di supporto

Come evidenziato da [16], l'intervento sulla dislessia richiede un approccio multidisciplinare che integri aspetti clinici ed educativi, con particolare attenzione al contesto italiano e alle sue specificità. Come suggerito da [stella2011] e [5], gli interventi più efficaci includono:

Interventi abilitativi e riabilitativi:

Logopedia: interventi mirati al potenziamento delle abilità fonologiche e metafonologiche.

Training di lettura: esercizi gradualmente che mirano ad automatizzare il processo di lettura.

Software e strumenti compensativi: sintetizzatori vocali, libri digitali (ebook), mappe concettuali digitali, applicazioni per la gestione dei testi.

Misure compensative e dispensative a scuola:

In Italia, la Legge 170/2010 [legge170] e successivi decreti attuativi hanno introdotto una serie di disposizioni che mirano a garantire il diritto allo studio degli alunni con DSA. Tra queste:

Strumenti compensativi: utilizzo di audio-libri, calcolatrice, sintesi vocale, mappe concettuali.

Misure dispensative: riduzione quantitativa dei compiti scritti, valutazione più attenta ai contenuti rispetto alla forma ortografica, tempi più lunghi nelle prove scritte.

Piano Didattico Personalizzato (PDP) un documento che definisce gli strumenti e le strategie da adottare in classe e a casa per favorire l'apprendimento.

Metodologie didattiche inclusive:

Apprendimento cooperativo: lavorare in piccoli gruppi, in cui gli alunni si sostengono a vicenda, favorisce l'inclusione di chi ha difficoltà.

Approcci multisensoriali: l'uso simultaneo di canali visivi, uditivi e cinestesici (es. Metodo Orton-Gillingham) può migliorare l'associazione tra lettere e suoni.

Mappe e schemi visivi: organizzano le informazioni in modo chiaro e aiutano la memorizzazione di concetti.

1.1.7 Aspetti emotivi e psicologici

La dislessia può avere un impatto significativo sull'autostima e sull'equilibrio emotivo della persona. Spesso, i bambini con dislessia sperimentano ripetuti insuccessi e frustrazioni a scuola, il che può tradursi in:

Ansia scolastica o da prestazione;
Demotivazione e rifiuto della lettura;
Sentimenti di inadeguatezza e bassa autostima.

È fondamentale che genitori e insegnanti sostengano emotivamente il bambino, valorizzandone i progressi e gli interessi, al fine di prevenire l'insorgenza di disturbi emotivi e comportamentali secondari.

1.1.8 Vita adulta e prospettive future

La dislessia non scompare con il termine della scuola dell'obbligo; può manifestare i suoi effetti anche nell'età adulta, specialmente in ambito universitario e lavorativo. Tuttavia, molte persone dislessiche sviluppano strategie di compensazione efficaci e, con il giusto supporto, possono raggiungere risultati eccellenti in vari campi professionali.

Università

Le università italiane prevedono spesso servizi di supporto per studenti con DSA (es. uffici di tutorato, servizi di counseling), che offrono strumenti compensativi e misure dispensative per gli esami.

Lavoro

Molti adulti dislessici sviluppano abilità peculiari (ad es. un'elevata creatività, capacità di problem solving, percezione visuo-spaziale) che possono rivelarsi un punto di forza in settori come design, arte, architettura, tecnologia. È importante che l'ambiente di lavoro sia flessibile e includa strategie di organizzazione e comunicazione adeguate.

1.1.9 In Breve

La dislessia è un disturbo complesso, che coinvolge le abilità di lettura e può influire significativamente sulla vita scolastica e professionale della persona. Grazie all'ampia ricerca neurobiologica e psicopedagogica, oggi si comprende molto meglio il disturbo e si dispone di strategie efficaci di diagnosi, intervento e supporto.

La sensibilizzazione sociale e la formazione degli insegnanti, unite a un quadro normativo di tutela (come la Legge 170/2010 in Italia), sono fondamentali per creare un contesto educativo inclusivo e rispettoso delle differenze individuali. Una presa in carico globale, che includa aspetti emotivi e relazionali, è essenziale per aiutare chi soffre di dislessia a sviluppare appieno le proprie potenzialità.

In conclusione, affrontare la dislessia in modo adeguato significa garantire a bambini, ragazzi e adulti l'opportunità di esprimere al massimo le proprie capacità, aiutandoli a gestire le difficoltà e a valorizzare le loro peculiarità. L'approccio integrato e la collaborazione fra scuola, famiglia e specialisti rappresentano la chiave per il successo formativo e personale dei soggetti con dislessia. Ed è questo il motivo fondante che ha portato alla realizzazione di questo Progetto di Tesi, che si vuole proporre come un approccio moderno all'aiuto ed il sostegno dei bambini, per accompagnarli in un percorso di convivenza, accettazione e mitigazione della condizione di dislessia. Accompagnando il potenziale bambino in un percorso di 200 giorni dove gli verranno proposti degli esercizi per conoscersi, capirsi e migliorarsi.

1.2 L'Avvento delle Nuove Tecnologie

1.2.1 Il concetto di accessibilità e l'impatto delle tecnologie

Secondo la classificazione ICF [33], l'accessibilità rappresenta un elemento fondamentale per garantire la piena partecipazione delle persone alla vita sociale e professionale. Nel contesto digitale, come definito dal W3C [29], questo concetto assume particolare rilevanza, specialmente considerando l'evoluzione delle tecnologie assistive e dei framework di sviluppo come quelli utilizzati in OpenDSA: Reading. L'accessibilità rappresenta, come definita dal W3C [**w3c_wai**], un elemento fondamentale nello sviluppo di tecnologie inclusive. Le linee guida WCAG [**wcag20**] [**wcag21**] hanno fornito un framework essenziale per lo sviluppo di contenuti digitali accessibili. Possiamo definirla l'insieme di caratteristiche che permettono a tutti gli utenti, compresi coloro con disabilità visive, uditive, motorie o cognitive, di fruire di un prodotto (digitale o fisico) in maniera completa. Prima dell'era digitale, le persone con disabilità incontravano barriere enormi nell'accesso all'informazione, ai servizi e alle opportunità di comunicazione. L'avvento delle nuove tecnologie – in particolare l'evoluzione di internet, dei dispositivi mobili, dei software di riconoscimento vocale e dei lettori di schermo – ha aperto scenari inediti di inclusione.

Esempi di tecnologie assistive:

Screen reader (es. JAWS, NVDA, VoiceOver su macOS/iOS, TalkBack su Android) per utenti non vedenti o ipovedenti.

Sintesi vocale e ingranditori di caratteri per facilitare la lettura dei contenuti.

Tecnologie di puntamento alternative (trackball, comandi oculari, switch) per chi ha difficoltà motorie.

Software di riconoscimento vocale (es. Dragon NaturallySpeaking) che consentono a utenti con problematiche motorie di interagire col computer tramite la voce.

Dispositivi hardware come tastiere Braille, display Braille, sensori e interfacce tattili.

Grazie alla digitalizzazione e alla crescente standardizzazione, gli sviluppatori possono integrare sin dall'inizio requisiti di accessibilità, facilitando l'uso di prodotti e servizi a una platea sempre più ampia di persone.

1.2.2 Le WCAG 2.0: linee guida per il web

Le Web Content Accessibility Guidelines (WCAG) [30] [31] rappresentano lo standard internazionale per l'accessibilità dei contenuti web. Questi principi hanno guidato anche lo sviluppo dell'interfaccia utente di OpenDSA: Reading, garantendo che l'applicazione sia utilizzabile da utenti con diverse abilità. Uno degli standard più importanti in ambito di accessibilità sul web è rappresentato dalle Web Content Accessibility Guidelines (WCAG). Sviluppate dal World Wide Web Consortium (W3C), in particolare dall'iniziativa WAI (Web Accessibility Initiative), esse offrono un quadro di riferimento completo per rendere i contenuti web utilizzabili da tutti.

1.2.3 Origine e struttura delle WCAG:

WCAG 1.0: pubblicate nel 1999, furono il primo tentativo di definire un insieme di raccomandazioni per sviluppare siti e applicazioni web accessibili.

WCAG 2.0: pubblicate nel dicembre 2008, hanno introdotto quattro principi fondamentali sintetizzati nell'acronimo POUR (Perceivable, Operable, Understandable, Robust), declinandoli in 12 linee guida e numerosi criteri di successo suddivisi in tre livelli di conformità (A, AA, AAA).

WCAG 2.1 (2018) e WCAG 2.2 (in fase di finalizzazione): versioni aggiornate per tenere conto delle nuove tecnologie (mobile, touch, gesture, etc.) e delle esigenze emergenti.

1.2.4 I quattro principi fondamentali (POUR)

Percepibile (Perceivibile): le informazioni e i componenti dell'interfaccia devono essere presentati in modo che siano percepibili da tutti i sensi disponibili (vista, udito, tatto), eventualmente tramite alternative (testo alternativo per le immagini, sottotitoli per i video, trascrizioni per i podcast, ecc.).

Utilizzabile (Operable): la navigazione e l'interazione con gli elementi dell'interfaccia devono essere tali da non costringere a usare un unico strumento di input. Devono prevedere la possibilità di usare tastiera, mouse, touch, comandi vocali, dispositivi speciali, ecc. Inoltre, i contenuti non devono causare reazioni fisiologiche (es. lampeggiamenti troppo veloci possono scatenare crisi epilettiche).

Comprensibile (Understandable): il contenuto deve essere chiaro, coerente e prevedibile. L'utente deve essere in grado di comprendere come interagire e come ricevere feedback sulle azioni compiute.

Robusto (Robust): i contenuti e i codici devono essere compatibili con tecnologie assistive e browser diversi, presenti e futuri. Ciò implica l'uso di standard aperti e la separazione tra contenuto e presentazione (HTML semantico, ARIA, CSS adeguato, etc.).

1.2.5 Adozione e benefici

L'adozione delle WCAG 2.0 (e successive) è auspicata da governi e organizzazioni internazionali per:

Garantire un principio di uguaglianza e inclusione;

Ridurre le barriere tecnologiche e l'esclusione digitale;

Migliorare la User Experience (UX) per un'utenza più ampia (anche non disabile, ad esempio utenti anziani o con connessioni lente).

1.2.6 Sviluppo di software e hardware “user friendly”

Le tecnologie digitali non si limitano ai siti web. È fondamentale progettare software e hardware inclusivi, secondo un principio di “progettazione universale” (Universal Design), che punti a rendere i dispositivi fruibili dal maggior numero possibile di persone, indipendentemente dalle loro abilità.

1.2.7 Sviluppo software inclusivo

Progettazione centrata sull'utente (UCD)

Coinvolgimento diretto di persone con diverse abilità fin dalle prime fasi di analisi dei requisiti.

Sessioni di “usability testing” con campioni di utenti reali, compresi coloro che utilizzano tecnologie assistive.

Iterazione continua: design → prototipo → test → miglioramenti →

ulteriore test.

1.2.8 Linee guida e toolkit

Aziende come Microsoft, Apple e Google offrono linee guida specifiche per lo sviluppo di app accessibili (per Windows, macOS/iOS e Android).

Framework e librerie front-end (es. Flutter - tecnologia usata all'interno del Progetto di tesi, Angular, React, Vue) includono componenti già ottimizzati per l'accessibilità (ad esempio, con attributi ARIA integrati).

1.2.9 Rispetto degli standard

Utilizzo di markup semantico (per il web), di etichette testuali, di controlli navigabili da tastiera, di testo alternativo per immagini, contrasti cromatici adeguati, ecc.

Inserimento di meccanismi di navigazione chiari (ad es. skip link) per aiutare gli utenti con screen reader o con mobilità ridotta.

1.2.10 Sviluppo hardware inclusivo

Design ergonomico: forme, pulsanti e materiali devono ridurre lo sforzo fisico e facilitare la presa, specialmente per chi ha problemi di destrezza manuale o di coordinamento.

Interfacce multisensoriali: l'uso di feedback aptici (vibrazioni), indicatori sonori o luminosi permette a persone con deficit uditivi o visivi di interagire con i dispositivi.

Compatibilità con dispositivi assistivi: per esempio, predisposizione di porte e protocolli che consentano a dispositivi medici o sistemi di controllo alternativi di integrarsi facilmente.

1.2.11 Esempi pratici di tecnologie e progetti inclusivi

Microsoft Inclusive Design: un insieme di principi e toolkit elaborati da Microsoft per progettare software (come Office, Windows) e servizi (come Xbox) che siano utilizzabili da tutti.

Apple Accessibility: Apple integra da anni funzioni di accessibilità a livello di sistema operativo (VoiceOver, Zoom, Switch Control, Live Captions, ecc.), dimostrando come l'hardware (touchscreen di iPhone e iPad) e il software (iOS, macOS) possano lavorare congiuntamente per facilitare l'uso da parte di persone cieche, ipovedenti o con limitazioni motorie.

Google Accessibility: Android e Chrome OS includono TalkBack (screen reader), comandi vocali, funzioni di ingrandimento e un ecosistema di app e servizi compatibili con standard WCAG e ARIA.

Le Distribuzioni Linux: integrano da sempre opzioni legate all'accessibilità, grazie soprattutto alla facoltà di poter manipolare tutto l'ambiente visivo a piacimento. Essi di fatti consentono di creare diversi ambienti che integrano automaticamente diverse agevolazioni in

termini di Accessibilità, tra cui: VoiceOver, facoltà di manipolare la grandezza del testo e delle icone a piacere, ecc. Di fatti, diversi Progetti software prendono spunto da queste straordinarie potenzialità degli Ambienti Grafici Linux per creare progetti software ad hoc gratuiti per il mondo scuola (So.di.Linux e UfficioZeroLinux OS Educational ne sono un esempio italiano lampante, il primo sviluppato dal CNR, il secondo da SIITE Srls).

Progetti open source: NVDA (Non Visual Desktop Access), uno screen reader gratuito per Windows, sviluppato con il contributo di una community internazionale.

Ognuno di questi ambienti software offre API e linee guida affinché gli sviluppatori di terze parti possano realizzare app e siti accessibili fin dalla fase di progettazione.

1.2.12 Benefici secondari dell’accessibilità

Un aspetto spesso sottovalutato è che progettare tecnologie accessibili non avvantaggia solo le persone con disabilità, ma migliora l’esperienza d’uso per tutti:

Migliore usabilità: un’interfaccia chiara e semplificata aiuta chiunque a orientarsi più rapidamente.

Approccio “mobile-first”: molte best practice di accessibilità (testi ben dimensionati, contrasti elevati, layout adattivi) sono fondamentali anche per dispositivi mobili.

SEO (Search Engine Optimization): un contenuto web con testo alternativo, titoli semantici e struttura ordinata risulta più facilmente

indicizzabile dai motori di ricerca.

Adattabilità: i contenuti robusti e compatibili con varie modalità di accesso durano nel tempo e non “invecchiano” rapidamente al cambiare delle tecnologie.

1.2.13 Legislazione e normative di riferimento

In molti Paesi, esistono leggi e normative che impongono o incoraggiano l’adozione di standard di accessibilità. Il quadro normativo europeo [9], americano [25] e italiano italiano [10] definisce:

Unione Europea:

Direttiva (UE) 2016/2102 sull’accessibilità dei siti web e delle applicazioni mobili degli enti pubblici.

European Accessibility Act (2019), che introduce requisiti di accessibilità per una vasta gamma di prodotti e servizi digitali.

Stati Uniti:

Section 508 del Rehabilitation Act (aggiornato nel 2017 in armonia con le WCAG 2.0 AA).

Americans with Disabilities Act (ADA), che impone l’accessibilità anche nei servizi digitali.

Italia:

Legge Stanca (Legge n. 4/2004), integrata da vari decreti attuativi, che prevede l’obbligo di accessibilità dei siti della Pubblica Amministrazione (e, in parte, dei soggetti privati con particolari requisiti di servizio al pubblico).

Questa cornice legislativa spinge le organizzazioni a implementare i principi delle WCAG e a progettare prodotti e servizi accessibili.

1.2.14 Prospettive future

L'avvento delle nuove tecnologie ha rivoluzionato il modo in cui le persone con diverse abilità possono accedere alle informazioni, interagire con gli altri e partecipare attivamente alla società. Le WCAG 2.0 – e le versioni successive come la 2.1 e la prossima 2.2 – rappresentano uno standard consolidato per rendere i contenuti web più inclusivi, mentre il concetto di universal design indica la direzione da seguire nello sviluppo di software e hardware realmente “user friendly”.

La sfida futura è andare oltre la semplice conformità normativa e puntare a un'accessibilità integrata in ogni fase di progettazione. Coinvolgere utenti con disabilità nei processi di sviluppo, adottare metodologie di progettazione partecipata e curare i dettagli dell'esperienza d'uso garantisce prodotti migliori per tutti e genera un impatto positivo sia dal punto di vista sociale che economico.

Le tecnologie continueranno a evolvere (intelligenza artificiale, realtà aumentata, robotica, Internet of Things) e sarà essenziale mantenere il focus sull'accessibilità, evitando di creare nuove barriere. Costruendo dispositivi e servizi universali, si garantisce la piena partecipazione di tutti i cittadini, promuovendo inclusione, pari opportunità e innovazione responsabile.

In sintesi, la tecnologia rappresenta oggi un potente alleato per le persone con problemi di accessibilità, a patto che sia sviluppata seguendo principi, standard e buone pratiche che tengano conto di tutte le possibili esigenze. L'innovazione, se accompagnata dall'impegno per l'inclusività, può davvero abbattere barriere secolari e garantire a tutti pari diritti e opportunità di partecipazione. Questo credo, è stato il

motivo fondante che ci ha fatto decidere di affrontare il problema della dislessia con le Nuove Tecnologie Emergenti.

1.3 L'Approccio alla Dislessia con le Nuove Tecnologie

1.3.1 L'Informatica ed i Moderni Approcci Tecnologici

L'Informatica, negli anni, ha sviluppato l'abilità camaleontica di potersi approcciare a diversi campi scientifici in modo efficace, proponendo soluzioni software ed ingegneristiche capaci di innovare e consentire di avere un punto di vista nuovo del problema che si stava affrontando inizialmente (si pensi alle soluzioni finanziarie, mediche, farmaceutiche, ecc).

L'Idea del Progetto di Tesi nasce da questa innata capacità, consentendo di unire nuove tecnologie come l'Intelligenza Artificiale, con l'uso di linguaggi di programmazione moderni e adatti alla creazione di Applicazioni nuove.

Poiché il progetto ha come focus principale la dislessia, deve concentrarsi sul trovare una soluzione stand-alone, semplice, intuitiva, di facile utilizzo e moderna che consentisse all'utente finale di poter usare l'applicazione, per sviluppare una maggiore confidenza con la propria condizione. Si è deciso di optare per l'uso di tecnologie come:

Software di riconoscimento vocale (Speech-to-text)

Trasformano la voce in testo, aiutando chi ha difficoltà a trascrivere o a scrivere correttamente e in modo fluido.

E di personalizzare la fruizione dei contenuti in modo da rispondere alle esigenze di ciascuno studente con dislessia. Ad esempio:

Modifiche del font e della spaziatura: alcuni font (come OpenDyslexic, Dyslexie Font o font ad alta leggibilità) sono studiati per rendere le lettere più distinguibili fra loro e ridurre gli errori di inversione o salto di riga.

Adeguamento del contrasto e dei colori: tablet, e-reader e smartphone consentono di regolare facilmente lo sfondo e la dimensione del testo, migliorando la percezione visiva.

Riduzione delle distrazioni: alcune applicazioni o estensioni del browser favoriscono la lettura a schermo intero, bloccando elementi che possano distrarre (banner, pop-up, ecc.).

In questo modo, lo studente con dislessia può configurare l'ambiente di studio secondo le proprie preferenze e necessità, minimizzando gli ostacoli legati alla lettura tradizionale.

Affidarsi a strumenti digitali non significa “barare” o aggirare il problema, ma potenziare le abilità dell'allievo e renderlo più indipendente:

Riduzione dell'errore ortografico: grazie ai correttori automatici, il ragazzo o l'adulto con dislessia può dedicarsi alla stesura di testi concentrandosi sul contenuto, senza essere ostacolato dagli errori ortografici.

Semplificazione della fase di studio: l'uso di audiolibri, ebook e app per la sintesi vocale permette di leggere (o ascoltare) più materiali, con meno fatica.

Aumento dell'autostima: sentirsi in grado di gestire compiti scolastici o lavorativi in autonomia incrementa la fiducia in sé stessi e

diminuisce lo stress.

1.3.2 Prospettive future

Utilizzare le nuove tecnologie per affrontare la dislessia non significa sostituire l'intervento didattico tradizionale o l'eventuale supporto logopedico/psicologico, ma integrare entrambi con soluzioni innovative. Ciò consente di:

Ridurre le barriere all'apprendimento, stimolando l'interesse e la partecipazione.

Sviluppare competenze metacognitive e strategie di studio personalizzate.

Promuovere un senso di autonomia e autostima, che incide positivamente sul benessere complessivo della persona.

In definitiva, l'investimento in tecnologie assistive e in un uso consapevole dei dispositivi digitali è uno dei modi più efficaci per supportare chi ha dislessia e massimizzare le potenzialità di ciascuno.

1.4 Il Ruolo della IA Generativa nel campo DSA

1.4.1 Cosa si intende per Intelligenza Artificiale generativa

Come delineato da [24], l'Intelligenza Artificiale generativa rappresenta una delle frontiere più promettenti nel campo dell'IA. Le ricerche condotte da organizzazioni come OpenAI [21] e DeepMind [8] hanno contribuito significativamente all'evoluzione di questa tecnologia, aprendo nuove possibilità anche nel campo del supporto educativo per persone con DSA. L'Intelligenza Artificiale generativa (o Generative AI) è un ramo dell'IA focalizzato sulla creazione di contenuti nuovi, come testi, immagini, suoni o video, partendo da modelli di apprendimento capaci di imitare e “ri-elaborare” i dati a cui sono stati esposti. Diversamente dai sistemi di AI che si limitano a classificare o riconoscere pattern (pattern recognition), la Generative AI genera appunto nuovi contenuti, spesso sorprendenti anche per gli stessi sviluppatori. Il concetto di Intelligenza Artificiale, teorizzato già da Turing [27], ha visto una svolta significativa con l'introduzione dei modelli Transformer [vaswani2017], che hanno rivoluzionato l'elaborazione del linguaggio naturale.

1.4.2 Le radici storiche dell'IA: dagli albori ai primi modelli generativi

Gli inizi dell'IA (anni '50-'70)

Alan Turing (1950): Nel suo articolo “Computing Machinery and Intelligence”, Turing propose il celebre “Test di Turing”, aprendo la

strada alle riflessioni sull'intelligenza delle macchine.

John McCarthy e Marvin Minsky (anni '50 [17]): Considerati tra i fondatori dell'IA come disciplina accademica, coniarono il termine “Intelligenza Artificiale” (1956) durante il Dartmouth Conference.

In questa fase, tuttavia, i primi algoritmi di IA erano di tipo simbolico e si concentravano su problemi di logica, algebra e puzzle. Non si parlava ancora di “creatività artificiale” o generazione di contenuti.

1.4.3 L'avvento dei sistemi conversazionali (anni '60-'70)

ELIZA (Joseph Weizenbaum, 1966): Il sistema ELIZA [32] rappresentò uno dei primi sistemi conversazionali, concepito per simulare uno psicoterapeuta rogersiano. Non era un modello generativo in senso stretto, ma aprì la porta a programmi in grado di elaborare risposte testuali (pur basandosi su pattern fissi).

Questi pionieri dimostrarono che le macchine potevano “produrre” contenuti linguistici, seppur in maniera molto limitata.

1.4.4 Il salto di qualità: reti neurali e apprendimento profondo

La rinascita delle reti neurali (anni '80-'90)

Reti neurali artificiali: Già dagli anni '40 e '50 erano state ipotizzate (McCulloch e Pitts), ma solo negli anni '80-'90, grazie a Backpropaga-

tion (metodo di addestramento efficiente), ricominciarono a diffondersi.

Tuttavia, la potenza computazionale e la disponibilità di dati rimanevano limitate, e il focus principale era sul riconoscimento di pattern piuttosto che sulla generazione di contenuti.

1.4.5 L’era del Deep Learning (anni 2000-2010)

Con l’aumento esponenziale della potenza di calcolo (GPU, cluster di server, cloud computing) e dei “big data” disponibili, gli algoritmi di Deep Learning iniziarono a ottenere prestazioni inedite:

Convolutional Neural Networks (CNN) – eccellenti per la visione artificiale.

Recurrent Neural Networks (RNN) – utili per l’elaborazione del linguaggio naturale.

È in questa fase che si prepara il terreno per i modelli generativi più avanzati.

1.4.6 La svolta generativa: dalle GAN ai Transformer

Generative Adversarial Networks (GAN)

Ian Goodfellow e colleghi (2014): pubblicano l’articolo “Generative Adversarial Nets”, presentando un’architettura rivoluzionaria.

L'introduzione delle GAN [13] e successivamente dei Transformer [28] consistono in due reti neurali (un “generatore” e un “discriminatore”) che si addestrano l'una contro l'altra, migliorando progressivamente la qualità dei contenuti generati (inizialmente, immagini).

Questa innovazione ha aperto la porta alla generazione di immagini realistiche, al deepfake e a un'intera gamma di applicazioni creative.

1.4.7 La rivoluzione dei Transformer

“Attention Is All You Need” ([**vaswani2017**]): il paper che ha introdotto l'architettura “Transformer”.

I modelli Transformer, a differenza delle RNN, possono processare in parallelo intere frasi o paragrafi, grazie al meccanismo di “attenzione”.

Nascono così i grandi modelli di linguaggio (Large Language Models, LLM) come BERT (Google), GPT (OpenAI) e successivi.

L'architettura Transformer ha reso possibile la Generative AI basata sul linguaggio ad altissimi livelli, permettendo la creazione di testi coerenti e di tipo “umano”.

1.4.8 L'impatto culturale della Generative AI

Epoca di “creatività aumentata”

La Generative AI non è più solo uno strumento tecnico, ma un fenomeno culturale:

Arte digitale e design: Strumenti come DALL-E, Midjourney, Stable Diffusion generano immagini artistiche e trasformano il ruolo di designer e artisti, che diventano “curatori” o “co-creatori” con l’IA.

Testi e contenuti online: Modelli come GPT (Generative Pretrained Transformer) sono in grado di scrivere articoli, riassunti, persino codice di programmazione, con un livello di fluidità mai visto prima.

Musica e audio: Sistemi in grado di comporre brani musicali o di imitare la voce umana con elevatissima fedeltà.

1.4.9 Rivoluzione sociale ed economica

La democratizzazione delle tecnologie: L’avvento di strumenti di Intelligenza Artificiale generativa accessibili tramite Internet ha reso queste tecnologie fruibili a un pubblico sempre più vasto (ad es. ChatGPT), rendendo la tecnologia alla portata di vaste masse.

Nuove questioni etiche: I deepfake e la generazione di contenuti potenzialmente disinformativi o ingannevoli hanno acceso dibattiti su responsabilità, privacy e plagio.

Nuovi mestieri e professionalità: “Prompt engineering” (l’arte di formulare richieste efficaci all’IA) e nuove figure professionali per supervisionare e integrare la produzione automatica di contenuti.

1.4.10 Perché è stata (ed è) rivoluzionaria

Accesso a capacità creative illimitate: Strumenti generativi permettono di estendere la creatività umana, fornendo spunti e prototipi in tempi rapidissimi.

Automazione di processi avanzati: L'IA non si limita più a compiti “ripetitivi”, ma può affrontare sfide di carattere creativo, linguistico e strategico.

Dibattito etico: La Generative AI ha posto questioni su originalità, copyright, veridicità dell'informazione e potere di manipolazione dell'opinione pubblica – tematiche di enorme portata culturale.

Cambiamento del paradigma di apprendimento: L'evoluzione dal Machine Learning “classico” a modelli Transformer e GAN ha accelerato la ricerca e l'innovazione in ambito IA, creando un impatto paragonabile alle più grandi rivoluzioni tecnologiche del secolo.

1.4.11 In Breve

Come evidenziato da [russell2021] e [18], l'evoluzione dell'Intelligenza Artificiale generativa racchiude diverse fasi: dalle prime intuizioni simboliche degli anni '50 alle moderne architetture neurali basate sui Transformer. Nel momento in cui i modelli di IA hanno iniziato a generare contenuti (testi, immagini, musica), ci si è trovati di fronte a una svolta che ha inciso profondamente non solo sulla ricerca tecnologica, ma anche sull'immaginario collettivo e sulla cultura di massa.

Oggi, la Generative AI rappresenta una nuova forma di creatività “aumentata” e solleva questioni etiche, sociali ed economiche senza

precedenti, spingendo l'umanità a interrogarsi sul confine tra l'umano e l'artificiale, tra l'originale e l'elaborato. È, a tutti gli effetti, una rivoluzione tecnologica e culturale destinata a imprimere una traccia indelebile nella storia.

Ragion per cui, visto l'incredibile potenziale rivelato, non poteva che essere una valida opzione d'uso nell'automazione di processi complessi come ad esempio il riconoscimento linguistico della lettura di un bambino dislessico. Processo complesso e spesso soggetto ad errore quando demandato ad uno speech-to-text tradizionale, ma che si è rivelato nel tempo molto efficace con l'ausilio e l'uso dell'intelligenza artificiale generativa, rinforzando l'idea che alla base di questi cambiamenti, i nuovi approcci e le nuove tecnologie, sono sempre più efficaci con il passare dei loro sviluppi nel tempo.

Capitolo 2

Tecnologie Utilizzate

2.1 Il Framework Flutter

Flutter è un *software development kit* (SDK) open source creato da Google[**flutterdev**] che permette lo sviluppo di interfacce utente (UI) nativamente compilate per diverse piattaforme, tra cui Android, iOS, web e desktop, il tutto partendo da un unico codice sorgente. Flutter ha dimostrato la sua efficacia non solo nello sviluppo di applicazioni mobile, ma anche nell'ambito dell'accessibilità, seguendo le linee guida definite dalla direttiva europea [9] sull'accessibilità dei contenuti digitali. Sin dal suo debutto, Flutter è stato considerato particolarmente *rivoluzionario* grazie a:

- L'utilizzo del linguaggio **Dart**, sviluppato anch'esso da Google.
- Un motore grafico ad alte prestazioni, chiamato *Skia*, che consente un rendering veloce e uniforme delle interfacce.
- Un approccio *reactive*, in cui la UI si aggiorna dinamicamente in risposta ai cambiamenti di stato.

- La filosofia “*write once, run anywhere*”, che riduce i tempi di sviluppo su più piattaforme.

2.2 Cenni storici

2.2.1 Il progetto Sky: i prodromi di Flutter

Come documentato in [15], Flutter si distingue per la sua architettura innovativa che permette di mantenere prestazioni native su tutte le piattaforme. La funzionalità di hot reload [12] ha rivoluzionato il processo di sviluppo, permettendo modifiche in tempo reale all’interfaccia. L’ecosistema di pacchetti [23] fornisce inoltre una vasta gamma di componenti riutilizzabili, molti dei quali sono stati integrati in OpenDSA: Reading per ottimizzare il processo di sviluppo. Flutter ha le sue radici in un progetto sperimentale di Google chiamato **Sky**, presentato per la prima volta nel 2015 durante il Dart Developer Summit. L’intenzione iniziale era di realizzare un motore grafico e un set di librerie in grado di rendere interfacce utente ad altissime prestazioni sul sistema operativo Android, partendo dal linguaggio Dart.

2.2.2 L’annuncio ufficiale di Flutter

È nel 2017, durante il Google I/O, che Flutter viene presentato al pubblico come SDK multiplatforma (inizialmente in alpha). Sin da subito, Google punta sull’idea di creare un framework UI “reactive” che potesse unificare lo sviluppo su diverse piattaforme (Android, iOS) mantenendo alte prestazioni e un controllo completo sul rendering.

2.2.3 Versione stabile e adozione in crescita

La prima versione stabile di Flutter (v1.0) avviene a dicembre 2018. L'evento viene celebrato come una pietra miliare per lo sviluppo mobile: offriva già allora la possibilità di scrivere app “nativamente compilate” in un unico codice sorgente, con un rendering engine personalizzato e reattivo.

2.3 Perché Flutter è stato rivoluzionario?

2.3.1 Unico framework per più piattaforme

Il mantra *cross-platform* non era nuovo, ma soluzioni precedenti come PhoneGap o Cordova non garantivano le stesse prestazioni di un'app nativa. L'implementazione del motore di rendering Skia in Flutter ha consentito un significativo miglioramento nell'esperienza utente, garantendo prestazioni native su diverse piattaforme, che disegna la UI in modo indipendente dalla piattaforma sottostante[[skiaDocs](#)].

A differenza di altri framework cross-platform, Flutter non si affida ai componenti nativi della piattaforma (come le UIViews su iOS o i widget Android). Al contrario, gestisce direttamente il rendering di tutti i componenti UI tramite il motore Skia, utilizzato anche da Google Chrome.

Benefici:

- Coerenza dell'aspetto grafico su tutte le piattaforme.
- Elevate prestazioni grazie al rendering su GPU.
- Controllo totale sul design e sui widget, senza passare da “ponti”

nativi.

2.3.2 Hot Reload e sviluppo rapido

Un elemento spesso lodato dagli sviluppatori è la funzione di *Hot Reload*: ogni modifica al codice sorgente viene istantaneamente riflessa sull'interfaccia (senza ricompilare o ricaricare l'app per intero). Questo approccio incrementa la produttività e riduce i tempi di *debug* e sperimentazione.

La funzionalità, oltre ad essere molto utile, rappresenta una *Rivoluzione* nel campo dello sviluppo Cross-Platform, in quanto con molto meno sforzo si potevano visionare importanti cambiamenti di design delle Interfacce Utente (UI - User Interface).

2.3.3 Widget personalizzabili e design coerente

La filosofia basata sui *widget* consente una vasta personalizzazione dell'aspetto delle app, dalla tipografia ai layout, con un controllo molto granulare. Ciò ha permesso a Flutter di competere efficacemente con soluzioni native, offrendo comunque la *material design experience* per chi sviluppa in ambiente Android.

2.4 La Rivoluzione di Flutter

A fronte di un mercato dinamico e competitivo, Flutter ha mostrato di poter offrire un'esperienza di sviluppo più rapida, coerente e flessibile rispetto ad altri strumenti *cross-platform*. La combinazione

di un linguaggio moderno come Dart, un potente motore di rendering, la ricchezza di *widget* e la *community* in espansione ha reso Flutter uno dei principali protagonisti nel panorama dello sviluppo di app multi-piattaforma.

2.4.1 Unificazione dello sviluppo cross-platform

Il principale punto di forza di Flutter è la vera convergenza tra le piattaforme mobili (Android e iOS), con un unico linguaggio (Dart) e un set di widget capaci di adattarsi o simulare l'aspetto “nativo”. A partire da Flutter 2.0, il supporto si è esteso anche a **Web**, **Windows**, **macOS** e **Linux**, facendo di Flutter un framework veramente “multi-platform”.

2.4.2 Prestazioni elevate

Grazie al compilatore AOT e al motore grafico Skia, Flutter riesce a garantire animazioni fluide a 60 FPS o 120 FPS (a seconda del dispositivo). Molti sviluppatori riferiscono che l'esperienza utente risulta paragonabile a quella di applicazioni scritte in modo nativo, se non superiore in certi aspetti grafici.

2.4.3 Community e ecosistema in rapida espansione

Sin dal rilascio ufficiale, Flutter ha attirato una vasta community, alimentata dal sostegno di Google. Esiste un fiorente ecosistema di pacchetti e plugin disponibili su pub.dev, e i continui aggiornamenti del

framework includono innovazioni come Flutter Web e Flutter Desktop.

2.4.4 Hot Reload e Produttività

Molti framework cross-platform erano già noti per facilitare lo sviluppo di app ibride (React Native, Xamarin, Ionic, ecc.), ma Flutter ha innalzato il livello di efficienza del flusso di lavoro. L'Hot Reload e l'approccio widget-based riducono i tempi di debug e di prototipazione in modo significativo, che ha cambiato anche la visione e la prospettiva dello sviluppo multi-piattaforma.

2.5 Evoluzione e impatto culturale/industriale

2.5.1 Adozione da parte di aziende e sviluppatori

Flutter è stato adottato da aziende di ogni dimensione per la realizzazione di app commerciali, prototipi o MVP (Minimum Viable Product). Un esempio emblematico è l'app di e-commerce di Alibaba, che ha sfruttato Flutter per la rapida creazione di interfacce ad alte prestazioni.

2.5.2 Convergenza tra mobile, desktop e web

Con Flutter 2.0 (2021) e le versioni successive, Google ha rafforzato la visione di un singolo toolkit capace di generare interfacce grafiche su qualunque dispositivo, dal browser web al PC desktop, passando per i dispositivi mobili. Questo ha influenzato la “filosofia cross-platform” nel settore IT, spingendo altri framework e tecnologie a migliorare il proprio approccio unificato.

2.5.3 Ruolo didattico e formativo

Flutter è utilizzato anche in contesti accademici e di formazione, per introdurre i concetti di sviluppo mobile e UI reattiva. La relativa semplicità del linguaggio Dart e la disponibilità di documentazione esaustiva ne fanno un’ottima scelta per chi vuole avvicinarsi al mondo delle app.

2.6 In Definitiva

La storia di Flutter è un esempio di come Google abbia saputo coniugare linguaggio di programmazione moderno (Dart), motore grafico ad alte prestazioni (Skia) e filosofia UI reattiva in un'unica soluzione. Il suo successo si deve soprattutto alla facilità d'uso e alle prestazioni notevoli, ma anche alla community che ne ha sostenuto lo sviluppo fin dalle prime versioni. Oggi Flutter rappresenta una delle tecnologie più rivoluzionarie in ambito cross-platform, offrendo un'unica codebase in grado di coprire l'intero spettro di dispositivi (mobile, web, desktop).

Queste speciali caratteristiche hanno fatto sì che venisse scelto come base per procedere con lo sviluppo del Progetto Software, che quindi ha come garanzia di poter essere eseguito su qualunque dispositivo e permette di poter essere mantenuto facilmente nel tempo, senza dover necessariamente considerare il dispositivo di utilizzo dell'utente o dello sviluppatore di software.

2.7 Dart: un Linguaggio Multiplatforma

2.7.1 La necessità di un nuovo linguaggio per il web

Le specifiche del linguaggio Dart [7] mostrano come sia stato progettato fin dall'inizio per supportare lo sviluppo di applicazioni moderne e scalabili. L'evoluzione da progetto Sky [14] all'attuale incarnazione del linguaggio riflette un percorso di continuo affinamento e ottimizzazione. All'inizio degli anni Duemila, le applicazioni web stavano divenendo sempre più complesse: si passava da pagine statiche a vere e proprie Single-Page Applications (SPA), arricchite da JavaScript e librerie emergenti. Google, in cerca di una soluzione per semplificare e velocizzare lo sviluppo, decise di investire in un linguaggio che potesse:

- Essere familiare per gli sviluppatori provenienti da linguaggi come C#, Java o JavaScript.
- Supportare la compilazione in JavaScript (per essere eseguito ovunque fosse disponibile un browser).
- Essere ad alte prestazioni, in modo da poter compilare in codice nativo (AOT, Ahead-of-Time) per scenari mobile o server.

2.7.2 Il lancio di Dart

Il linguaggio Dart è stato annunciato per la prima volta da Google durante la GOTO conference 2011 a Aarhus, in Danimarca. Gli ideatori principali furono Lars Bak e Kasper Lund, noti per il loro lavoro sulle macchine virtuali (V8, usata in Chrome).

- Obiettivo dichiarato: sostituire (o quantomeno offrire un'alternativa) a JavaScript, ottimizzando la produttività degli sviluppatori e le prestazioni delle web app di grandi dimensioni.

2.8 Principi fondamentali e prime versioni

2.8.1 Flessibilità e portabilità

Dart, fin dall'inizio, ha cercato di conciliare due esigenze:

- Semplicità sintattica: ispirata a linguaggi tipizzati staticamente (C, Java, C#) e dinamici (JavaScript).
- Portabilità sul web: la capacità di compilare in JavaScript era cruciale per garantire compatibilità con qualsiasi browser (Dart2js).

2.8.2 Dart 1.0 (2013)

La versione 1.0 di Dart è stata rilasciata nel 2013, consolidando la sintassi base, il modello a classi, la tipizzazione opzionale (in origine), e introducendo il concetto di isolati (Isolates) per la gestione della concorrenza senza lock sharing.

Isolates: ogni isolato è un processo leggero che comunica con gli altri tramite passaggio di messaggi, evitando problemi di threading tipici di altri linguaggi.

2.9 Evoluzione e caratteristiche rivoluzionarie

2.9.1 Dall'interpretazione alla compilazione AOT

Uno dei punti di svolta di Dart è stato il passaggio alla compilazione Ahead-of-Time per usi mobile e server-side. Questo ha consentito di:

- Ridurre i tempi di esecuzione rispetto all'interpretazione pura.
- Rendere più agevole lo sviluppo di app nativamente compilate, con prestazioni comparabili alle applicazioni scritte in linguaggi come Java o Swift.

2.9.2 Dart 2.0 (2018) e la tipizzazione forte

Con Dart 2.0, rilasciato nel 2018, il linguaggio ha adottato una tipizzazione forte e sicura (sound type system). Ciò ha incrementato:

Affidabilità del codice (meno errori di runtime).

Ottimizzazioni da parte del compilatore.

Qualità degli strumenti di sviluppo (autocompletamento, refactoring).

2.9.3 Dart DevTools e Hot Reload

Dart ha introdotto strumenti integrati (Dart DevTools) per il debug, l'analisi delle prestazioni e l'ispezione dello stato a runtime. Inoltre, nella combinazione con il framework Flutter, Dart permette l'utilizzo dell'Hot Reload, il caricamento “a caldo” delle modifiche, con un

miglioramento sostanziale nei cicli di sviluppo UI.

Sebbene l'Hot Reload sia spesso associato a Flutter, è la macchina virtuale Dart che lo rende possibile grazie alla compilazione JIT (Just-in-Time) durante lo sviluppo.

2.10 Dart e Flutter: un legame strategico

2.10.1 La Simbiosi

Il linguaggio Dart è divenuto popolare a livello globale soprattutto grazie a Flutter, il framework di Google per lo sviluppo di app multiplatforma. L'unione di un motore grafico (Skia), un'architettura reattiva e la compilazione nativa (Dart AOT) ha mostrato come Dart non fosse soltanto un “linguaggio sperimentale”, ma una colonna portante di un ecosistema in espansione.

2.10.2 I vantaggi di Dart nel contesto mobile

Performance: grazie ad Ahead-of-Time, le app Flutter scritte in Dart possono raggiungere i 60/120 fps in modo costante.

Developer Experience: la combinazione di Dart DevTools, l'editor Visual Studio Code (o Android Studio) e l'Hot Reload fa sì che l'iterazione sia molto rapida.

2.11 Come Dart è stato rivoluzionario

Versatilità multiplatforma: Dart può essere compilato per il Web (JavaScript), per app mobili (AOT nativo) e persino per il backend (via Dart VM).

Modello di concorrenza sicuro: l'uso degli isolati riduce la complessità tipica dei thread condivisi, facilitando la scrittura di codice concorrente. Performance competitiva: benché nato per il web, Dart ha dimostrato di poter competere con linguaggi consolidati anche in ambito di sviluppo mobile e server-side.

Ecosistema in rapida crescita: la popolarità di Flutter ha spinto la community a creare pacchetti, plugin e soluzioni attorno al linguaggio, accelerandone la diffusione.

Cambiamento di paradigma per il web: inizialmente, Dart voleva sfidare JavaScript come linguaggio nativo per i browser. Anche se i principali browser (a parte Chrome) non hanno integrato una VM Dart nativa, la strategia “compile to JS” ha permesso di mostrare un modello più rigoroso e tipizzato per le web app.

2.12 Adozione industriale e prospettive

Aziende come Adobe, Alibaba e Canonical hanno iniziato a integrare Dart e Flutter in prodotti reali, dimostrando che il linguaggio può trovare spazio anche in ambienti enterprise.

L'evoluzione continua, con l'introduzione di funzionalità come il Null Safety (Dart 2.12) e l'adozione di nuovi paradigmi di programmazione reattiva (streams, async/await).

2.13 In Conclusione

Dart è un linguaggio che ha saputo adattarsi alle esigenze di un mercato in rapida evoluzione: nato come potenziale alternativa a JavaScript, si è poi trasformato in un caposaldo per lo sviluppo di applicazioni multiplatforma (con Flutter) e non solo. La sua architettura basata su isolati, la compilazione AOT, il type system solido e la forte integrazione con strumenti di sviluppo lo rendono una soluzione innovativa e produttiva per sviluppatori di ogni livello.

In definitiva, Dart ha contribuito ad abbattere barriere tecnologiche e a spingere l'idea che un linguaggio, ben progettato, possa essere efficiente e facile da apprendere, pur mantenendo un occhio di riguardo alle best practice nel campo della programmazione concorrente e reattiva.

Questo e la sua nativa simbiosi con Flutter, lo hanno reso il Linguaggio di Programmazione di riferimento dell'intero progetto di Tesi.

2.14 Il Modello VOSK

2.14.1 Origini e contesto storico: L'Evoluzione dei sistemi di riconoscimento vocale

VOSK si basa sulle fondamenta tecnologiche del toolkit Kaldi [22], un framework open source per il riconoscimento vocale. Come documentato in [1] e nel repository ufficiale [2], VOSK è stato progettato per offrire un'implementazione efficiente e facile da integrare del riconoscimento vocale, caratteristiche che lo hanno reso la scelta ideale per OpenDSA: Reading. I sistemi di riconoscimento vocale hanno una lunga storia, a partire dai progetti di ricerca degli anni '70 e '80 (ad esempio, il sistema Dragon e i primi prototipi di IBM), basati su modelli Hidden Markov Models (HMM). Nel corso dei decenni, l'intelligenza artificiale e, in particolare, il Machine Learning e il **Deep Learning**, hanno permesso di migliorare notevolmente l'accuratezza dei modelli di Speech-to-Text.

2.14.2 Kaldi e la nascita di VOSK

Uno dei framework open source più noti in ambito di riconoscimento vocale è Kaldi, nato nel 2009 all'interno di progetti di ricerca accademica (principalmente presso la Johns Hopkins University), con l'obiettivo di fornire strumenti flessibili per la ricerca e lo sviluppo di sistemi di ASR (Automatic Speech Recognition).

VOSK si basa in parte sulle tecnologie e sulle conoscenze sviluppate con Kaldi, ma nasce come progetto finalizzato a rendere più semplice l'integrazione del riconoscimento vocale offline e in tempo reale in applicazioni concrete.

È mantenuto da Alpha Cephei, un'organizzazione che sviluppa soluzioni basate sul riconoscimento vocale open source.

2.15 Caratteristiche tecniche di VOSK

2.15.1 Funzionamento offline

Una delle peculiarità più apprezzate di VOSK è la possibilità di svolgere il riconoscimento vocale senza necessitare di una connessione internet. Questo contrasta con molte soluzioni di speech-to-text proprietarie (Google Cloud, Amazon Transcribe, Apple Dictation, ecc.) che, in gran parte, si basano su infrastrutture cloud.

Vantaggi:

Privacy e sicurezza: i dati vocali restano in locale, non vengono inviati a server esterni.

Flessibilità: l'applicazione può funzionare in contesti privi di rete (aeroporti, località remote, veicoli) o in situazioni in cui la latenza deve essere bassa.

2.15.2 Supporto multilingua e modelli pre-addestrati

VOSK mette a disposizione modelli pre-addestrati per numerose lingue (tra cui inglese, italiano, spagnolo, russo, tedesco, francese e molte altre). Questi modelli possono essere utilizzati immediatamente oppure raffinati con dati specifici per un dominio verticale (ad esempio, vocabolari tecnici).

2.15.3 Architettura modulare

Dal punto di vista architetturale, VOSK fornisce:

API di alto livello (in C++ e in altri linguaggi con binding, come Python, Java, JavaScript e persino Swift) per l'integrazione rapida.

Componenti di decodifica basati su reti neurali addestrate su grandi dataset audio e ottimizzate per l'esecuzione in tempo reale su CPU (senza obbligo di GPU).

2.15.4 Licenza Open Source

VOSK è rilasciato sotto licenza Apache 2.0. Ciò consente la libera consultazione, modifica e distribuzione del codice sorgente, anche per scopi commerciali, a patto di rispettare i termini della licenza.

2.16 L'approccio open source: vantaggi rispetto alle soluzioni closed

Trasparenza e ispezionabilità: Chiunque può studiare il codice, verificare la qualità degli algoritmi e proporre modifiche.

Personalizzazione: Le aziende o i singoli sviluppatori possono aggiungere funzionalità, ottimizzare i modelli per settori specifici (sanità, industria automotive, etc.).

Autonomia dal fornitore: Non si è vincolati a un provider cloud o a una licenza proprietaria che può cambiare termini e costi.

Comunità e innovazione collaborativa: L'ecosistema open source favorisce la condivisione di soluzioni, miglioramenti e patch da parte

di un'ampia base di sviluppatori.

2.17 L'integrazione di VOSK in Flutter

2.17.1 Plugin e pacchetti di terze parti

Per integrare VOSK in un'applicazione Flutter, si ricorre generalmente a plugin che espongono le API di VOSK al codice Dart. Esistono diverse librerie open source che forniscono un'implementazione del riconoscimento vocale basato su VOSK, come ad esempio:

flutter_vosk o eventuali fork simili, dove viene creato un bridge (piattaforma nativa) fra i binding di VOSK e l'app Flutter.

Nel nostro caso specifico abbiamo dovuto fare riferimento ad una versione locale di `flutter_vosk`, ma nel parleremo più approfonditamente nella sezione dedicata più avanti.

2.17.2 Processo di integrazione nel Progetto

- Installazione del plugin: aggiunta della dipendenza `flutter_vosk` (o simile) nel file `pubspec.yaml`.
- Configurazione dei modelli: download di uno dei pacchetti di lingue (ad esempio, per l'italiano, il modello `vosk-model-it`), da includere nei file di risorse dell'app.
- Inizializzazione: nel codice Dart, si crea un'istanza del riconoscitore (`Recognizer`) indicando il path del modello.

- Gestione degli stream audio: l'app deve acquisire l'audio dal microfono e passarlo a VOSK in tempo reale, ottenendo i risultati del riconoscimento (trascrizioni parziali e finali).
- Analisi e uso dei dati: i testi decodificati possono quindi essere utilizzati per funzionalità come comandi vocali, compilazione di moduli o ricerca vocale interna all'app.

Nel nostro caso specifico, abbiamo dovuto effettuare la sovrascrittura della dipendenza del plugin flutter, con una sua versione locale (che ho dovuto modificare) per poter rendere la app compatibile a tutti gli ecosistemi Linux, che sono spesso eterogenei e non hanno una uniformità delle proprie dipendenze. La patch inserita bypassa questa problematica, permettendo l'inizializzazione del plugin in tutte le casistiche (per ulteriori approfondimenti leggere la sezione dedicata, più avanti nel testo).

2.17.3 Vantaggi di VOSK su Flutter

Completamente offline: non necessità di backend remoto per trascrivere l'audio.

Prestazioni: se ben configurato, VOSK può raggiungere latenze molto basse in tempo reale anche su dispositivi mobili di fascia media.

Personalizzazione: la possibilità di “tarare” il modello sulle esigenze dell'app (parole specifiche, dizionari personalizzati) consente di incrementare l'accuratezza rispetto ad alternative generiche.

2.18 Perché VOSK è rivoluzionario

Ci sono degli Elementi Tecnici che rendono VOSK Rivoluzionario rispetto ai suoi competitor (anche commerciali) e lo hanno di fatto reso un valido candidato per il Progetto di Tesi:

Riconoscimento vocale offline di elevata qualità: Mentre molte soluzioni di speech-to-text necessitano di cloud computing, VOSK permette di eseguire l'intero processo su device locali, aprendo la strada a un nuovo livello di privacy e autonomia.

Integrazione cross-platform: Grazie alle API e ai binding in diversi linguaggi, VOSK può essere adottato su un'ampia gamma di piattaforme (mobile, desktop, embedded). **Comunità open source in crescita:** Il progetto vede continui contributi, fix e miglioramenti da parte di sviluppatori di tutto il mondo.

Flessibilità: L'integrazione con framework moderni come Flutter rende più semplice la creazione di app vocali o assistenti virtuali con un'interfaccia reattiva e personalizzabile.

2.19 Riflessioni sul futuro e prospettive

Il Progetto VOSK dimostra di avere una chiara direzione dettata dalle potenzialità e scelte espresse nel tempo, che possiamo racchiudere in:

Espansione del supporto linguistico: Sebbene VOSK già offra un buon set di lingue, il potenziamento di modelli addestrati con reti neurali più avanzate (ad es. Transformer-based, Conformer, ecc.) potrebbe ulteriormente aumentare la precisione. **Possibile estensione a scenari IoT:** Le soluzioni offline di VOSK lo rendono adatto a dispositivi embedded che operano in ambienti dove la connettività non è garantita. **Ulteriore ottimizzazione per mobile:** Con l'evoluzio-

ne di Flutter e di hardware mobile, VOSK potrà ridurre i consumi di risorse (CPU, RAM), ampliando le sue applicazioni.

2.20 In Conclusione

VOSK rappresenta uno dei sistemi di riconoscimento vocale open source più completi e versatili. Storicamente, è erede di un lungo percorso iniziato con Kaldi, e incarna la filosofia della condivisione e dell'innovazione collaborativa. La sua integrazione con Flutter, resa possibile grazie a plugin che semplificano l'interazione tra Dart e le API di VOSK, offre una soluzione potente per creare applicazioni mobile e desktop capaci di comprendere la voce offline.

Il fatto che sia open source (licenza Apache 2.0) lo rende estremamente flessibile e trasparente, consentendo a sviluppatori e organizzazioni di utilizzare, modificare e distribuire la tecnologia con meno vincoli rispetto alle alternative proprietarie. Sul piano tecnologico, l'approccio offline e la community attiva lo rendono un elemento rivoluzionario nella trasformazione dei servizi vocali, spingendo sempre più in là la frontiera del cosiddetto “edge computing” e dell'IA distribuita.

Presentando queste potenzialità enormi, la possibilità di poter essere utilizzata offline, la sua essenza Open Source e la sua integrazione con la tecnologia Flutter, hanno fatto sì che venisse scelta senza alcuna esitazione per il Progetto di Tesi, che di fatto, ha come motore proprio la tecnologia VOSK.

Capitolo 3

Processo di Sviluppo del Software

3.1 La Scelta del Cross-Platform

La scelta implementativa di utilizzare il Cross-Platform (cioè la possibilità di lanciare la App su qualsiasi dispositivo) è nata dalla necessità di poter abbracciare un pubblico quanto più ampio possibile ed allo stesso tempo, alla necessità di non volersi preoccupare troppo del sistema utente, ma solo sullo sviluppo delle Funzionalità della App stessa.

Queste motivazioni, hanno portato alla ricerca curata di tecnologie che potessero dialogare tra loro e tra possibili progetti Futuri dell'Università di Torino stessa, nella volontà di poter un giorno creare una Suite di Applicazioni in grado di aiutare studenti (e non) afflitti da specifiche problematiche DSA.

Come spiegato nel capitolo precedente, la scelta delle tecnologie utilizzate non è stata casuale, ed ha portato ad una visione a lungo ter-

mine della vita della app (e del suo possibile aggiornamento futuro). Di fatti, è possibile reperire il progetto sulla Piattaforma di Rilascio del Software GitHub al link: <https://github.com/LorenzoElSalserito/DyslexiaApp>. Rilasciato con licenza G.P.L. 3.0 come Progetto FOSS.

3.2 Design del Software

3.2.1 Alberatura del Progetto

```
DislexiaApp
├── android
├── build
├── ios
├── lib
│   ├── assets
│   │   ├── exercises
│   │   ├── fonts
│   │   └── icon
│   ├── config
│   ├── models
│   ├── old
│   ├── screens
│   ├── services
│   ├── Tesi_LaTeX
│   ├── utils
│   ├── widgets
│   └── main.dart
├── linux
├── local_vosk_flutter
├── macos
├── saves
├── test
├── web
├── windows
├── .flutter-plugins
├── .flutter-plugins-dependencies
├── .gitignore
├── .metadata
├── analysis_options.yaml
├── build.sh
├── LICENSE
├── linuxdeploy-x86_64.AppImage
├── pubspec.lock
├── pubspec.yaml
├── README.md
└── thesis_project.iml
```


3.2.2 Struttura del Progetto

La Struttura del Progetto è stata impostata a "livelli", implementando un vero e proprio Progetto dove le schermate (screens e widgets) richiamano dei servizi (services) che però, invece di interpellare un DataBase, richiamano (o creano nel caso della creazione di un profilo) dei file di testo, che vengono poi immagazzinati nei modelli di dati quando utilizzati (models). Gli oggetti che non sono più utilizzati, sono stati inseriti nella cartella old (perché funzionanti, ma attualmente non richiesti - per possibili integrazioni future), mentre le configurazioni di progetto sono nella cartella config. In utils troviamo librerie di utilità che non hanno una classificazione specifica, ma che vengono richiamate in più punti del progetto.

3.2.3 Origini del concetto di architettura a livelli

Il termine architettura a livelli (in inglese layered architecture) fa riferimento a un modo di organizzare il software in strati (o livelli), ognuno con responsabilità distinte e ben definite, che comunicano tra loro seguendo relazioni gerarchiche. L'idea di fondo è semplificare la complessità, suddividendo il sistema in componenti coesi e separati, riducendo così le dipendenze incrociate e migliorando la manutenibilità.

Pur essendo nato come concetto in ambienti enterprise (Java, .NET, ecc.), questo paradigma è adottato con successo anche nel mondo mobile (iOS, Android, Flutter), soprattutto per gestire la complessità di progetti crescenti.

3.2.4 Principi fondanti

Separazione delle responsabilità (Separation of Concerns): ogni livello si occupa di un aspetto logico diverso (presentazione, business logic, accesso ai dati, ecc.).

Comunicazione unidirezionale: tipicamente, uno strato superiore può dipendere da uno strato inferiore, ma non viceversa (evitando dipendenze circolari).

Manutenibilità e testabilità: avendo livelli autonomi, si possono testare e modificare parti del sistema senza intaccare l'intero progetto.

3.3 Struttura tipica di un progetto Flutter

3.3.1 Il contesto Flutter

In Flutter, la cartella "lib/" ospita gran parte del codice sorgente scritto in Dart. In un progetto layered, gli sviluppatori organizzano le directory per riflettere tale suddivisione in livelli, spesso aggiungendo cartelle come:

screens: Schermate (o pagine) dell'app, spesso associabili alla parte di presentation layer.

widgets: Componenti riutilizzabili di UI (bottoni, card, sezioni grafiche), in parte riconducibili al presentation layer (con scopi più generici).

services o business logic / controllers: Logica applicativa e/o di dominio (spesso considerabili come application layer).

models: Classi che rappresentano oggetti di dominio, entità o strutture dati, avvicinabili al domain layer di un'architettura più evoluta.

utils: Funzioni di supporto (formattazioni, parser, costanti).

config: Impostazioni globali, definizioni di ambiente, costanti di configurazione.

repositories (opzionale): Se l'app interagisce con fonti dati (API REST, database, file system), ci si appoggia a uno strato “repository” per incapsulare la persistenza e le query.

Nota: non esiste un obbligo formale per i nomi di cartella, ma in generale si notano scelte convergenti. Ad esempio, è comune sostituire “services” con “data” o “controllers”, oppure includere la logica di rete in un'apposita cartella “network”.

3.4 Descrizione dei livelli

La letteratura classica (Fowler, Bass e colleghi) identifica alcuni livelli fondamentali, che in ambito Flutter possono essere mappati come segue:

Presentation Layer (Livello di presentazione)

Responsabilità: Gestione dell'interfaccia utente, layout, interazione con l'utente finale (touch, click), validazione basilare dei dati in input (se serve a migliorare l'UX).

In Flutter: è tipicamente rappresentato da screens, widgets, e talvolta pages.

Comunicazione: Riceve eventi dall'utente e li invia al livello di business logic; rappresenta graficamente i dati ricevuti dai livelli inferiori.

Application/Business Logic Layer (Livello di logica o “servizi”)

Responsabilità: Implementa la logica dell'applicazione, gestisce i flussi di dati tra presentation e data (nel caso di pattern come BLoC o Provider, incapsula stati e comportamenti). **In Flutter:** spesso si concretizza in cartelle come services, controllers, providers, blocs. Qui si definiscono classi di gestione per singoli casi d'uso (es: AuthenticationService, UserController, ecc.).

Comunicazione: Riceve chiamate da Presentation, elabora la richiesta (ad esempio, invocando repository di dati), e restituisce risultati pronti per la UI.

Data/Infrastructure Layer (Livello di accesso ai dati)

Responsabilità: Accedere a fonti dati esterne (API REST, database locale, file system), formattare e persistere i dati. Questo livello è quello più vicino a “dettagli infrastrutturali”.

In Flutter: spesso gestito da “repository” (cartelle repositories o services se implementano anche la parte di persistenza).

Comunicazione: Espone metodi per eseguire CRUD (Create, Read, Update, Delete) su varie fonti dati. Deve restituire a services/controllers entità o modelli coerenti, gestendo eccezioni e errori di rete.

Domain Layer (opzionale e più tipico in “Clean Architecture”)

Responsabilità: Racchiudere la logica di dominio pura, rappresentando le regole fondamentali dell'applicazione.

In Flutter: si potrebbe situare in models (dove ogni entità e la relativa logica rimangono indipendenti da dettagli di presentazione o infrastruttura).

Comunicazione: Il domain layer è “chiamato” dal business layer per eseguire le regole di business; a sua volta, dipende da astrazioni (interfacce) per accedere ai dati, in uno stile di inversione delle dipendenze.

3.5 Vantaggi specifici dell'approccio

3.5.1 Manutenibilità e testabilità

Avere livelli chiari permette di testare (unit test o widget test) ogni porzione di codice in modo isolato. Ad esempio, i test sui servizi non coinvolgono l'interfaccia, e viceversa. Se un domani si decide di cambiare una libreria di networking, le modifiche rimangono localizzate allo strato services (o repositories).

3.5.2 Scalabilità

Le app Flutter possono crescere da prototipi a prodotti enterprise. Una struttura a livelli consente di aggiungere nuove funzionalità senza “sporcare” il codice esistente. Invece di ammassare tutto in un singolo file o in poche cartelle, si rispettano le responsabilità di ciascuna sezione.

3.5.3 Principio di “Single Responsibility”

Ogni classe, funzione, file, tende a rispettare il Single Responsibility Principle (SRP), un cardine della programmazione orientata agli oggetti. In un’architettura a livelli, questo risulta più naturale, perché i servizi (logica) non si confondono con la UI, e i modelli (dati) non si mescolano con la grafica.

3.6 Criticità e limiti

Possibile overhead: Nei progetti piccolissimi (demo o prototipi iniziali), suddividere in troppi livelli può risultare un carico eccessivo per lo sviluppatore (un numero elevato di file e cartelle).

Complessità di navigazione: Se non ben documentato, un progetto troppo segmentato può confondere chi deve trovare rapidamente un pezzo di logica.

Rischio di duplicazione: Se non vengono ben definiti i confini tra i livelli, si può incorrere in duplicazioni di codice o “confusione” (p.es.: quando una stessa funzione di validazione si trovi sia in utils che in screens).

3.7 Pattern collegati in Flutter

3.7.1 BLoC (Business Logic Component)

Molto popolare in Flutter, definisce uno strato (o più bloc) responsabile di gestire eventi e stati, separando la UI dalla logica. Si integra bene nel concetto di architettura a livelli, dove il BLoC funge da “ponte” tra presentation e data.

Riferimento: Felix Angelov. (2018). “Bloc - An Architect’s Approach to Flutter Development.” [Medium / GitHub repos]

3.7.2 Provider / Riverpod

Alternative o evoluzioni del concetto di InheritedWidget, semplificano l’accesso a servizi e stati condivisi nell’app. Anch’esse si sposano con un approccio a livelli, riducendo la dipendenza tra UI e logica.

Riferimento: Remi Rousselet. (2019). “provider” package. <https://pub.dev/packages/provider>

3.7.3 Clean Architecture

Alcuni sviluppatori Flutter applicano la “Clean Architecture” di Robert C. Martin (Uncle Bob). Essa formalizza in modo rigoroso la separazione tra presentation, domain e data (o infrastructure). È una variante più rigida di un’architettura a livelli, con l’obiettivo di ottenere un elevato grado di disaccoppiamento (inversione delle dipendenze).

Riferimento: Martin, R.C. (2012). Clean Architecture: A Craftsman’s Guide to Software Structure and Design. Prentice Hall.

3.7.4 Uso dell’Architettura a Livelli

Nel caso specifico, si è deciso di procedere con una architettura a Livelli, in modo tale da avere una struttura modulare ibrida, che fosse

coerente con tutti i componenti del progetto e mantenendo solida la sua struttura e semplice la sua manutenibilità nel tempo. Questo inoltre concederà la possibilità in futuro di integrare nuove funzionalità e di espandere le possibilità della App, permettendo ad altri sviluppatori di accrescerne i contenuti, le idee e di espanderne le potenzialità.

3.8 L'Uso della Gamification

3.8.1 Origini e Definizione

La gamification è un concetto nato nei primi anni 2000, quando si è iniziato a studiare come gli elementi dei videogiochi potessero essere applicati a contesti aziendali e sociali. Il termine è stato formalizzato da Nick Pelling nel 2002, ma è diventato popolare solo intorno al 2010.

3.8.2 Meccaniche e Dinamiche di Gioco

Meccaniche di Gioco

Le meccaniche sono gli strumenti concreti che rendono il sistema di gamification efficace. Alcuni esempi includono:

Punti (Cristalli nel nostro caso): Ricompense numeriche che rappresentano il progresso dell'utente.

Trofei: Riconoscimenti visivi per traguardi raggiunti.

Livelli: Sistemi di progressione che sbloccano contenuti man mano che l'utente avanza.

Classifiche (Non Presenti): Elenchi competitivi che motivano gli utenti confrontandoli tra loro.

Sfide e Missioni: Obiettivi specifici che stimolano il coinvolgimento.

Progress Bar: Indicatori visivi dello stato di avanzamento.

Ricompense Virtuali: Bonus o vantaggi che migliorano l'esperienza dell'utente.

Dinamiche di Gioco

Le dinamiche rappresentano gli aspetti psicologici che rendono la gamification efficace:

Competizione: Spinge gli utenti a migliorarsi confrontandosi con altri.

Collaborazione: Stimola il lavoro di squadra e l'aiuto reciproco.

Narrativa (Non Presente): Crea un'esperienza immersiva attraverso una storia avvincente.

Autonomia: Dà agli utenti la libertà di scegliere come progredire. **Sco-po:** Motiva gli utenti facendoli sentire parte di qualcosa di significativo.

3.8.3 Design Pattern Usati

Capitolo 4

Sviluppo

4.1 Le Meccaniche di Gioco nella User Journey

Per le Meccaniche che avrebbero regolato l'intero gioco, si è deciso di seguire un flusso di azioni, traguardi e conseguenze ben preciso, in modo tale da mantenerlo stimolante e fluido nel tempo.

Il Flusso di Gioco inizia con l'atterrare nella schermata principale e creare il nostro "Profilo Giocatore". Qualora si voglia avere più profili, questo è possibile e la gestione di essi (eventuale eliminazione ed uso) sono determinati dall'Interfaccia Grafica semplice ed intuitiva.

Una Volta creato il Profilo del giocatore, verremo immediatamente indirizzati nella Pagina di Gioco, dove avremo un riepilogo delle nostre statistiche, e potremo visionare: lo Store dei Trofei, le Sfide o tornare al Menù Iniziale dei profili attraverso gli appositi pulsanti. Qualora stessimo giocando da più tempo, se faremo il Login per più di un giorno consecutivo, ci verranno regalati 10 Cristalli di Bentornato, per stimolare uno dei principi della Gamification.

I Cristalli

I Cristalli sono la "moneta di gioco", è possibile utilizzarli per comperare trofei, che abbelliscono con nomi Fantasiosi, la Schermata di Gioco affiancando il nome del Profilo, questo per rendere omaggio ed onore al Giocatore, che tanto si è impegnato per poter raggiungere l'ambito risultato.

4.1.1 I Trofei

I Trofei infatti, comperabili nell'apposito Store, saranno sì disponibili, ma molto difficili da comperare. Inizialmente, un Nuovo Giocatore che Crea un Profilo, parte con il Titolo "Novizio", in quanto ancora non ha "dimostrato il suo valore" migliorando le proprie capacità. Soltanto giocando si ha la possibilità di incrementare il numero di cristalli acquisiti e poter quindi accumularne un numero abbastanza alto da poter permettere al Profilo di comprare i Trofei.

4.1.2 La Durata di Gioco

Questi meccanismi aiutano ad innescare nell'utente la voglia di migliorarsi (ottenendo quindi maggiori cristalli e trofei) e consentendo nello stesso tempo di migliorare il suo grado di apprendimento. La durata complessiva del gioco è calcolata in giorni, ammettendo che l'utente giochi senza mai sbagliare, ci vorrebbero 5 giorni per superare ogni livello, per un totale di 4 livelli (espandibili fino a 7). Quindi 20 giorni totali per superare tutti i livelli la prima volta. Una volta raggiunto questo obiettivo però, si innesca il "New Game+", una

meccanica che aumenta con un moltiplicatore il numero di Cristalli guadagnati, facendo ricominciare il gioco dal Livello 1. Il moltiplicatore, aumenta fino a 10 volte (10 New Game Plus attivati) prima di bloccarsi, pertanto, facendo tutto perfetto, bisognerebbe trascorrere almeno 200 giorni (senza espansioni) per poter dire di aver raggiunto la Fine del gioco. Un periodo comunque molto lungo.

4.1.3 Gli Esercizi

Gli esercizi vengono presi da un pool di file di testo casualmente, il file di testo viene selezionato in base al livello. Più esso è alto, maggiore sarà la difficoltà degli esercizi proposti. Il gioco si ricorda dei primi 20 esercizi per cercare di non essere ripetitivo, ma poi si resetta per non affaticare la memoria (soprattutto dei dispositivi mobili).

Gli Esercizi vengono proposti in batterie da 5, in modo da tenere alta la *suspense* e mantenere attivo ed attento il Giocatore. Un Feedback visivo comparirà al termine di ogni esercizio completato e facendoci il riassunto di come siamo andati sul singolo esercizio e nel totale della batteria al suo termine.

4.1.4 Il Profilo

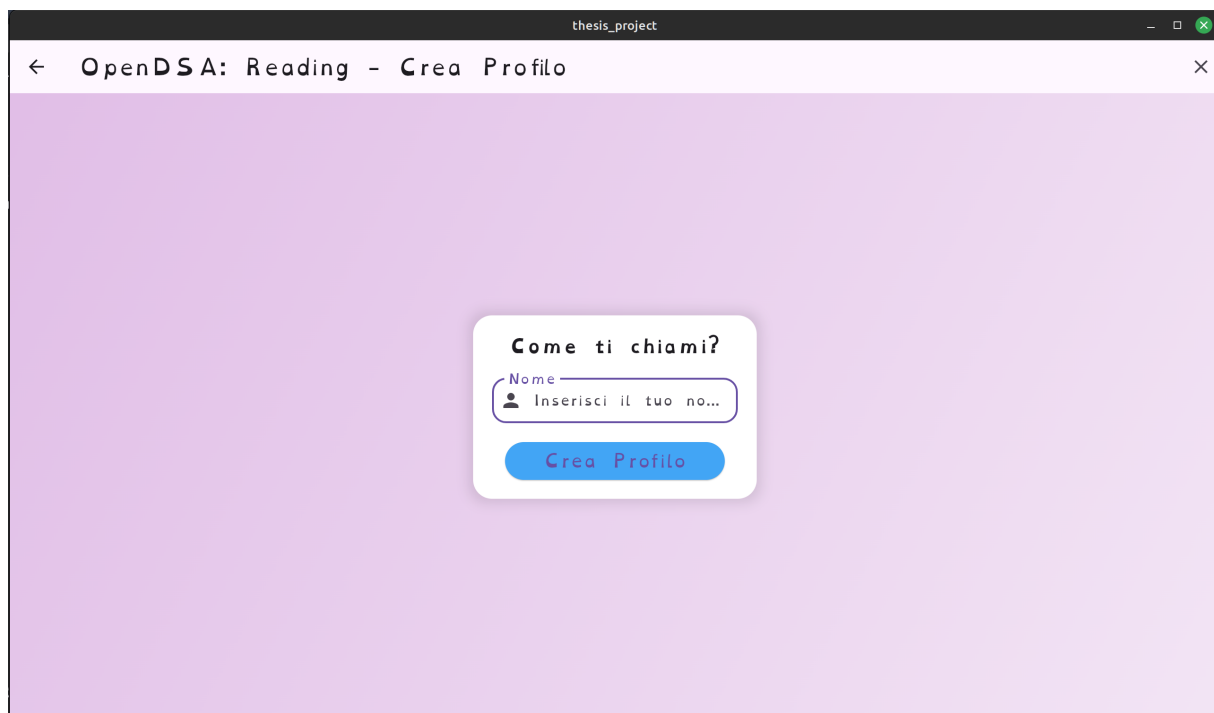
Tuttavia l'elemento più importante dell'intera struttura resta il Profilo Giocatore. Esso oltre a tenere traccia dei progressi di gioco ottenuti, è anche il fulcro del gioco stesso. Infatti, ogni profilo viaggia autonomamente, e conserva i dati delle partite passate, definendo un vero e proprio percorso nell'utilizzo dell'applicazione e imponendosi come pilastro dei dati del singolo giocatore ed individuo.

¹Per i Dettagli dei Trofei, delle Sfide, degli Achievements, ecc. Vedere la sezione Dedicata nell'appendice A

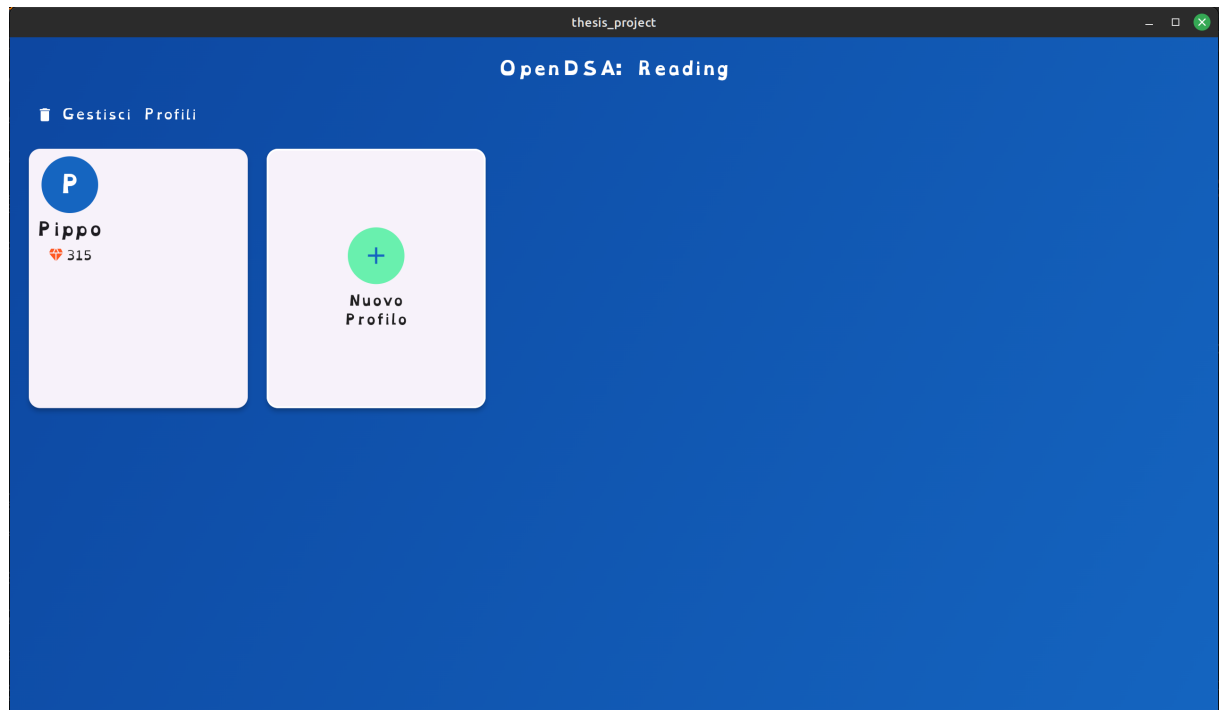
4.2 L'Interfaccia Grafica

4.2.1 La Schermata di Avvio

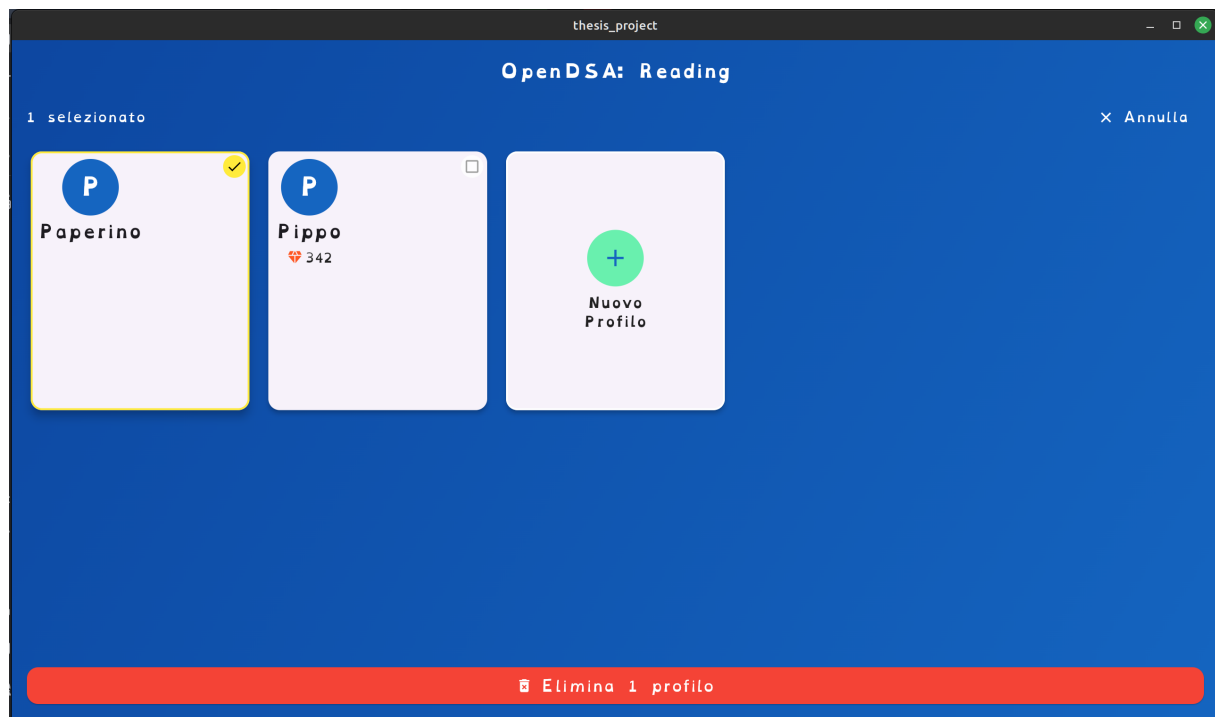
La Schermata di Avvio della App ci farà atterrare nella Schermata di Selezione del Profilo, che quando avviata per la prima volta, contiene solo la Card per Creare un Nuovo Profilo. Già dal primo impatto, possiamo notare forti contrasti di colore (ho selezionato queste tonalità seguendo le WCAG ed le tonalità descritte nel libro: "L'Arte del Colore" di Johannes Itten) ed il Font OpenDyslexic (fondamentale in tutto il progetto).



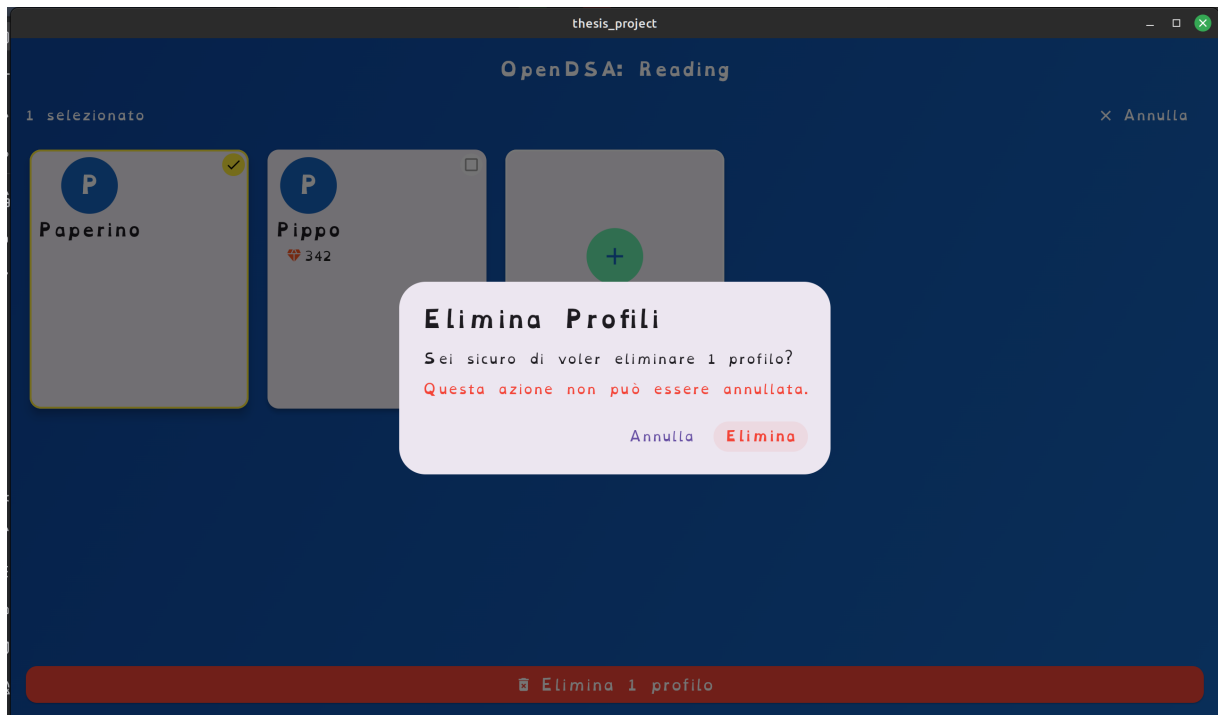
4.2.2 La Selezione Del Profilo



Poiché ho avuto modo di usare l'Applicazione, in questo momento è presente il Profilo di prova "Pippo" (nome consueto per i test informatici). Tuttavia, in alto a sinistra, è possibile notare un pulsante "Gestisci Profili", questo pulsante consentirà di abilitare delle piccole checkbox nelle card dei Profili per poterli eventualmente eliminare.

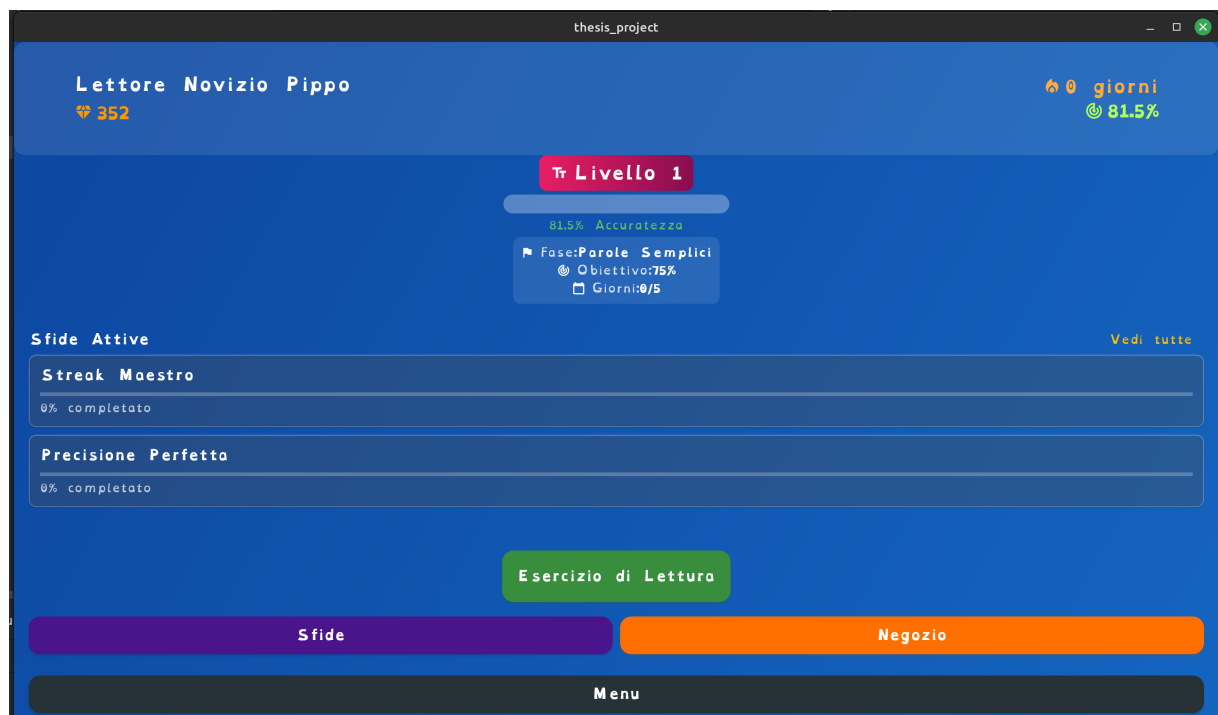


Alla scelta del Profilo da eliminare, un popup ci intimerà ad accertarci della nostra azione, in quanto irreversibile. Questo meccanismo è stato pensato per prevenire eventuali errori o per gestire eventuali ripensamenti che l'utente potrebbe avere nella User Journey della App.



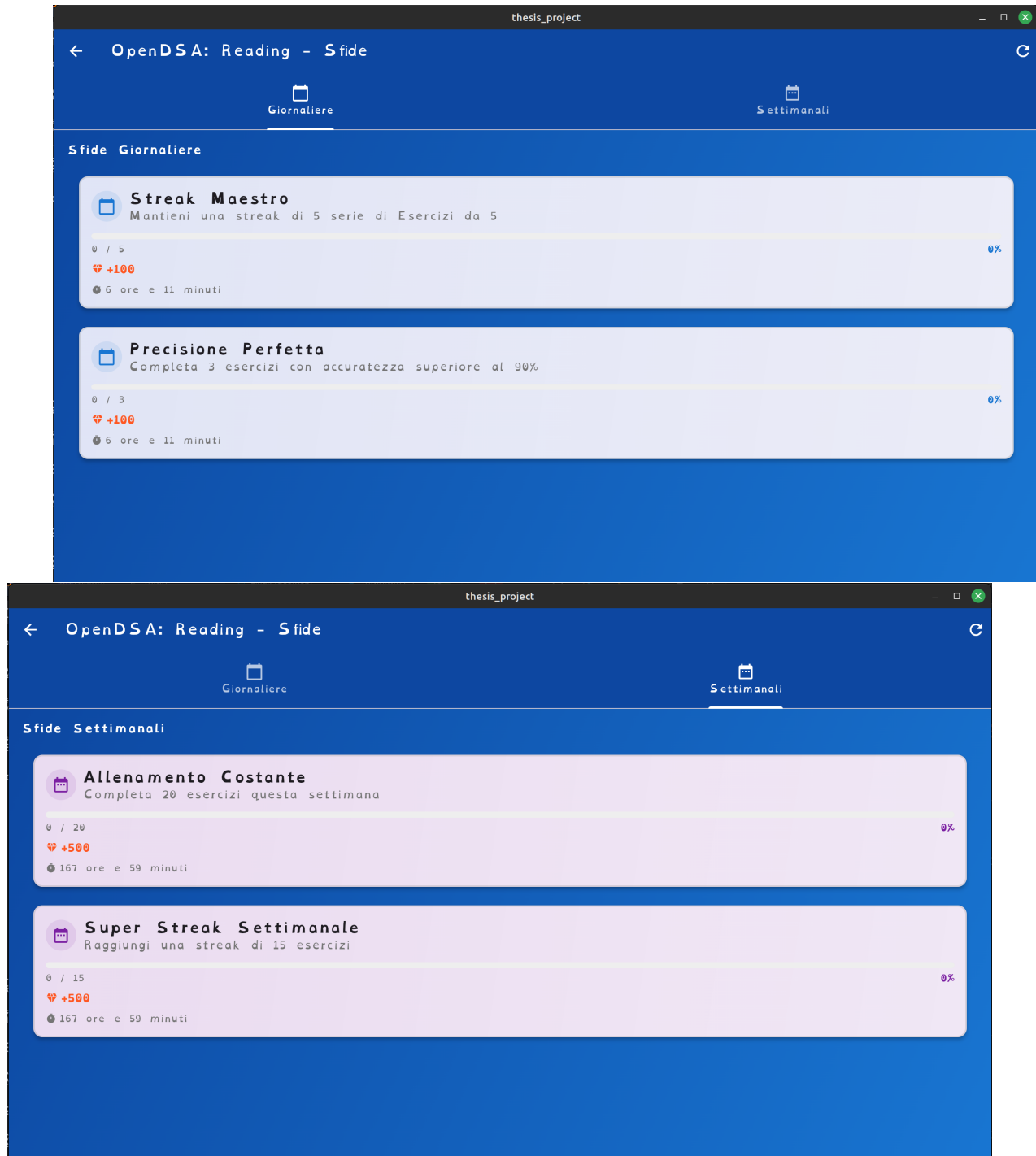
Una volta deciso di voler giocare, verremo Catapultati all'interno della Schermata di Gioco, con il riepilogo del nostro Profilo (Obiettivi da raggiungere, Cristalli accumulati, barra di progressione del Livello, numero di giorni per cui abbiamo giocato, Progresso delle nostre sfide, ecc). La schermata è piuttosto intuitiva e colorata per dare enfasi alle varie sfaccettature del Gioco.

4.2.3 La Schermata di Gioco Principale



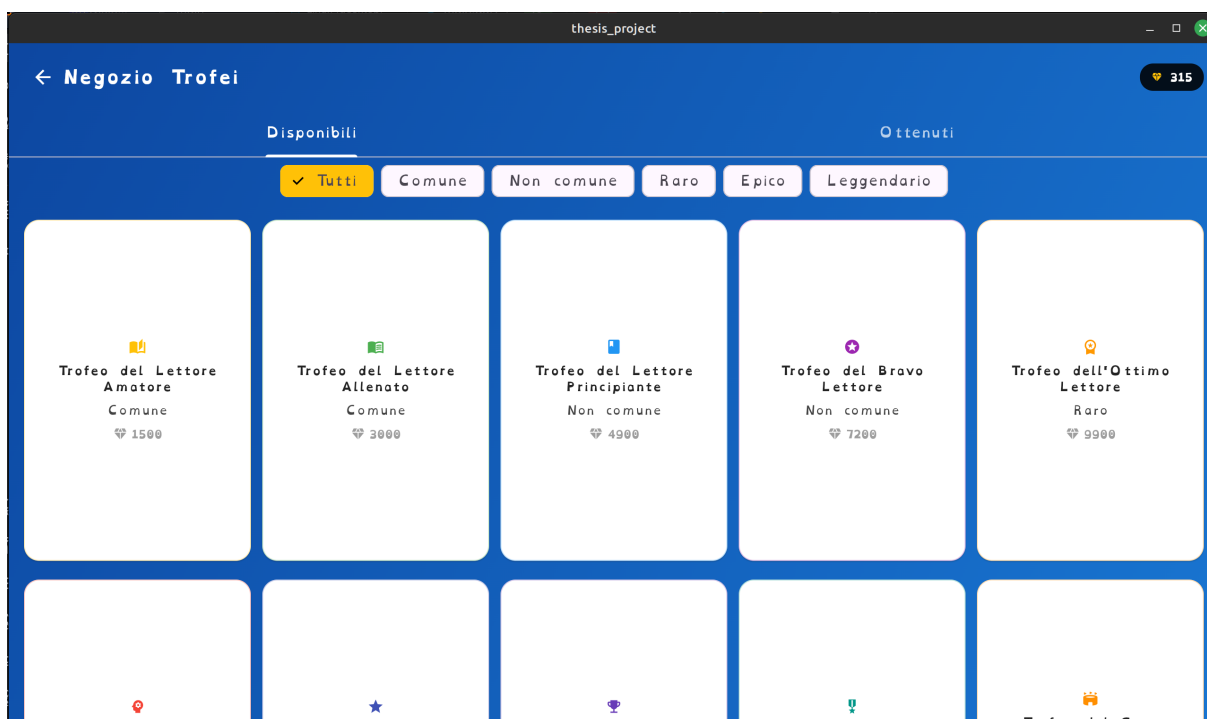
Attraverso il Pulsante "Sfide" potremo accedere alla prima parte della struttura di Gamification implementata, che aiuterà a mantenere alto l'interesse verso l'uso della app (L'obiettivo delle Sfide è autoesplicativo).

4.2.4 Le Sfide

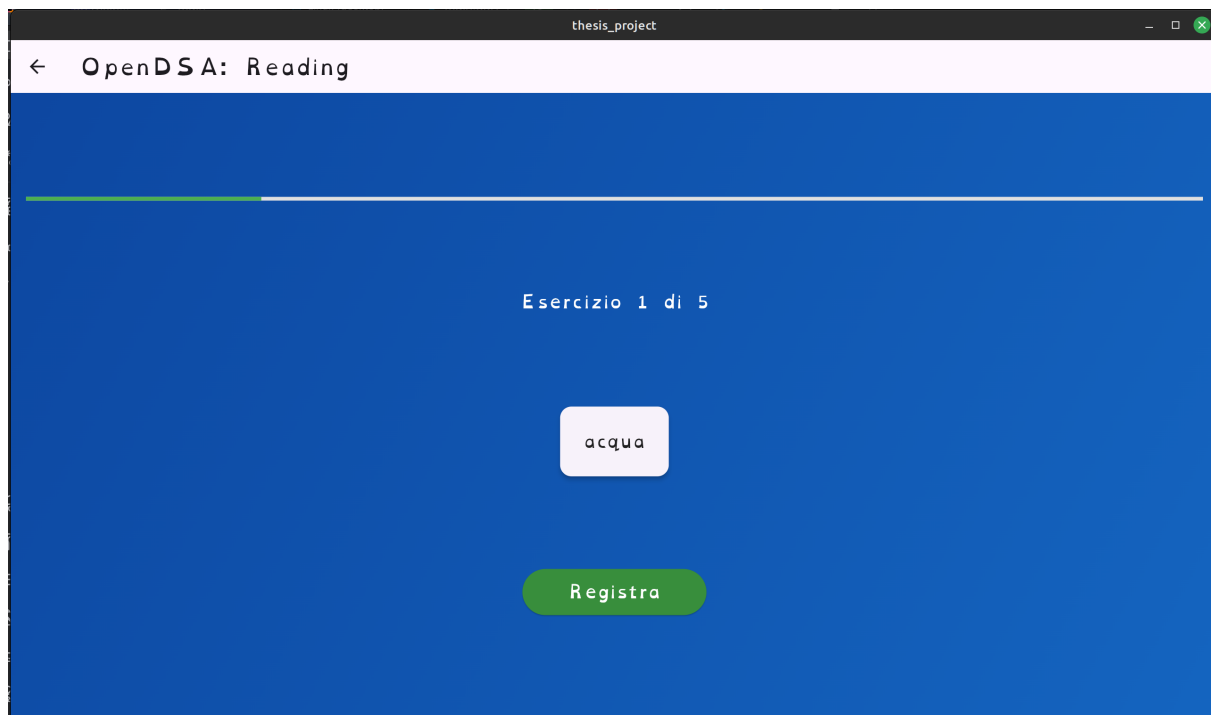


4.2.5 I Trofei

Se decidessimo di entrare nello Store dei Trofei invece, possiamo notare le fantasiose e colorate Card dei Titoli che possiamo acquistare. Di fatti, il Titolo si andrà a sostituire all'attuale "Lettore Novizio" con titoli più entusiasmanti che potranno essere comprati attraverso i cristalli accumulati nelle varie sessioni di gioco.



4.2.6 Gli Esercizi



Attraverso il Pulsante "Esercizio di Lettura" invece, ci verranno proposti un set di 5 esercizi di fila, che dovremo risolvere registrando un file audio attraverso l'uso del microfono. Questo poi, verrà elaborato attraverso il VoskService (Servizio che si appoggia alla AI di VOSK) e attraverso le Ottimizzazioni degli errori comuni della Dislessia (vedere elenco in Appendice B) e l'Algoritmo di Riconoscimento della Distanza di Levenshtein, elaborerà l'audio in testo e fornirà un output riconoscitivo.

4.3 Algoritmo di Levenshtein

La *distanza di Levenshtein*, nota anche come *distanza di edit*, è una misura di similarità tra due stringhe. In altre parole, essa quantifica il

numero minimo di operazioni elementari richieste per trasformare una stringa nell'altra. Questa metrica è stata introdotta per la prima volta nel 1965 dal ricercatore russo Vladimir Levenshtein.

Le operazioni che vengono prese in considerazione sono:

- **Cancellazione** di un carattere,
- **Sostituzione** di un carattere con un altro,
- **Inserimento** di un carattere.

Ad esempio, per trasformare la stringa "bar" nella stringa "biro" sono necessarie due operazioni:

1. Sostituzione di 'a' con 'i': "bar" $\rightarrow$ "bir"
2. Inserimento del carattere 'o': "bir" $\rightarrow$ "biro"

Quindi, la distanza di Levenshtein tra "bar" e "biro" è pari a 2.

4.3.1 Uso e applicazioni

La distanza di Levenshtein trova impiego in molteplici ambiti:

- **Correttori ortografici:** per suggerire parole candidate quando l'utente commette un errore di digitazione;
- **Riconoscimento OCR e vocale:** per confrontare stringhe di riconoscimento con voci di un dizionario;
- **Analisi di similarità in Data Mining:** per raggruppare o indicizzare stringhe simili;
- **Ricerca di pattern approssimata:** utile quando si cercano corrispondenze imprecise (ad es. motori di ricerca di nomi o keyword);

- **Comparazione di sequenze biologiche:** sebbene spesso si adottino varianti più complesse, la distanza di Levenshtein può costituire un punto di partenza per il confronto di brevi sequenze di DNA o proteine.

4.3.2 Proprietà e varianti

Oltre alla definizione classica, esistono varianti dell'algoritmo di Levenshtein che tengono conto di altre operazioni, come ad esempio lo *scambio di due caratteri adiacenti* (la cosiddetta *distanza di Damerau-Levenshtein*). Tuttavia, la forma originaria rimane fra le più utilizzate per semplicità ed efficienza computazionale.

4.3.3 Formalizzazione della distanza

La distanza di Levenshtein tra due stringhe A e B è definita come il *numero minimo* di operazioni elementari (inserimenti, cancellazioni o sostituzioni di un singolo carattere) per convertire la stringa A nella stringa B .

Esempi

- Distanza tra "casa" e "cassa":
 - "casa" $\rightarrow$ "cassa" (inserimento di 's'),
quindi distanza = 1.
- Distanza tra "gatto" e "matto":
 - "gatto" $\rightarrow$ "matto" (sostituzione di 'g' con 'm'),
quindi distanza = 1.

- Distanza tra "sarto" e "sorto":
 - "sarto" $\rightarrow$ "sorto" (sostituzione di 'a' con 'o'),
 quindi distanza = 1.

Naturalmente, esempi più complessi richiederanno un numero maggiore di passi.

4.3.4 Descrizione dell'algoritmo

Un metodo classico per calcolare la distanza di Levenshtein tra due stringhe `str1` e `str2` di lunghezze `lenStr1` e `lenStr2`, rispettivamente, si basa sulla costruzione di una matrice bidimensionale di dimensione $(\text{lenStr1} + 1) \times (\text{lenStr2} + 1)$.

- **Dimensione della matrice:** la dimensione è $(n + 1) \times (m + 1)$ dove $n = \text{lenStr1}$ e $m = \text{lenStr2}$;
- **Interpretazione:** ogni cella $d[i, j]$ della matrice rappresenta la distanza di Levenshtein tra i primi i caratteri di `str1` e i primi j caratteri di `str2`.

L'algoritmo si basa su un approccio *dinamico*:

1. Inizializzazione: le prime righe e le prime colonne della matrice vengono valorizzate con indici crescenti (i costi necessari per trasformare una stringa di lunghezza i in una stringa vuota o viceversa);
2. Ricorrenza: per ogni coppia di indici (i, j) si controlla se i caratteri corrispondenti delle due stringhe sono uguali. In caso positivo, si aggiorna la cella $d[i, j]$ con il valore di $d[i - 1, j - 1]$.

In caso negativo, si calcola la minima distanza tra l'operazione di *cancellazione*, *inserimento* o *sostituzione* e si somma 1 come costo dell'operazione.

Lo pseudocodice di riferimento è il seguente:

```
1 int LevenshteinDistance(char str1[1..lenStr1], char str2[1..lenStr2
  ])
2 // d[i1, i2] conterra' la distanza di Levenshtein tra
3 // i primi i1 caratteri di str1 e i primi i2 caratteri di str2.
4 declare int d[0..lenStr1, 0..lenStr2]
5 declare int i1, i2, cost
6
7 for i1 from 0 to lenStr1
8   d[i1, 0] := i1
9 for i2 from 0 to lenStr2
10   d[0, i2] := i2
11
12 for i1 from 1 to lenStr1
13   for i2 from 1 to lenStr2
14     if str1[i1 - 1] = str2[i2 - 1] then
15       cost := 0
16       d[i1, i2] := d[i1 - 1, i2 - 1]
17     else
18       cost := 1
19       d[i1, i2] := minimum(
20         d[i1 - 1, i2] + 1,          // cancellazione
21         d[i1, i2 - 1] + 1,          // inserimento
22         d[i1 - 1, i2 - 1] + cost // sostituzione
23       )
24
25 return d[lenStr1, lenStr2]
```

Listing 4.1: Pseudocodice dell'algoritmo di Levenshtein

4.3.5 Esempio di calcolo dettagliato

Si consideri il confronto fra "cane" e "gatto":

- "cane" → "gane" (sostituzione di 'c' con 'g');

- "gane" $\rightarrow$ "gatne" (inserimento di 't' prima di 'n');
- "gatne" $\rightarrow$ "gatto" (sostituzione di 'n' con 't').

In questo caso la distanza calcolata è 3.

4.3.6 Analisi dell'algoritmo

4.3.7 Complessità

La complessità temporale dell'algoritmo classico è $O(n \cdot m)$, dove:

- $n = \text{lenStr1}$
- $m = \text{lenStr2}$

La complessità in memoria è anch'essa $O(n \cdot m)$, essendo necessaria la costruzione di una matrice di dimensioni $(n + 1) \times (m + 1)$.

4.3.8 Ottimizzazioni

Sono possibili varie ottimizzazioni, tra cui:

- **Spazio:** esistono versioni che riducono l'utilizzo di memoria salvando solo la riga corrente e la riga precedente della matrice (o di colonna in colonna), abbassando la complessità spaziale a $O(\min(n, m))$.
- **Pruning:** per applicazioni in cui è noto a priori un limite superiore alla distanza massima d'interesse, si possono adottare metodi di *taglio* (pruning) per interrompere l'algoritmo quando viene superata tale soglia.

La distanza di Levenshtein rappresenta un riferimento fondamentale nello studio delle stringhe in vari campi dell'informatica e dell'elaborazione dei dati. Il suo algoritmo, pur nella semplicità, costituisce un ottimo esempio di utilizzo della programmazione dinamica e fornisce risultati utili in tutte quelle situazioni in cui è importante stimare quanto due stringhe differiscano tra loro.[4]

Questo Algoritmo viene inglobato all'interno del `vosk_service.dart` (classe in Appendice B) assieme al codice fonetico, in modo da garantire che il risultato venga elaborato contestualmente agli errori di pronuncia classici che possono essere commessi (per pronuncia, dialetto o influenza regionale). Di seguito le funzioni che si occupa di questo passaggio:

```
1  // Mappa per errori comuni (usata nelle funzioni di similarita')
2  static const Map<String, List<String>> _commonDyslexicConfusions =
3      {
4          'b': ['d', 'p'],
5          'd': ['b', 'q'],
6          'p': ['q', 'b'],
7          'q': ['p', 'd'],
8          'm': ['n', 'w'],
9          'n': ['m'],
10         'a': ['e'],
11         'e': ['a'],
12         's': ['z'],
13         'z': ['s'],
14         'f': ['v'],
15         'v': ['f'],
16     };
17 double _calculatePhoneticSimilarity(String s1, String s2) {
18     String phonetic1 = _getPhoneticCode(s1);
19     String phonetic2 = _getPhoneticCode(s2);
20     return _calculateLevenshteinSimilarity(phonetic1, phonetic2);
21 }
```

```

22
23 String _getPhoneticCode(String text) {
24     var result = text.toLowerCase();
25     final Map<String, String> phoneticRules = {
26         'chi': 'ki',
27         'che': 'ke',
28         'ghi': 'gi',
29         'ghe': 'ge',
30         'gn': 'ñ',
31         'gl': 'ɹ',
32         'sc': 'ʃ',
33         'qu': 'k',
34         'gu': 'g',
35         'z': 'ts',
36         'zz': 'ts',
37     };
38     for (var entry in phoneticRules.entries) {
39         result = result.replaceAll(entry.key, entry.value);
40     }
41     result = result.replaceAll(RegExp(r'([aeiou])\1+'), r'$1');
42     return result;
43 }
44
45 double _calculateTextSimilarity(String text1, String text2) {
46     if (text1 == text2) return 1.0;
47     if (text1.isEmpty || text2.isEmpty) return 0.0;
48     const double levenshteinWeight = 0.4;
49     const double phoneticWeight = 0.3;
50     const double confusionWeight = 0.3;
51     double levenshteinScore = _calculateLevenshteinSimilarity(text1,
52         text2);
53     double phoneticScore = _calculatePhoneticSimilarity(text1, text2
54         );
55     double confusionScore = _calculateConfusionSimilarity(text1,
56         text2);
57     return (levenshteinScore * levenshteinWeight) +
58         (phoneticScore * phoneticWeight) +
59         (confusionScore * confusionWeight);
60 }
61
62 double _calculateConfusionSimilarity(String s1, String s2) {
63     if (s1.length != s2.length) return 0.0;
64     int matchCount = 0;
65     for (int i = 0; i < s1.length; i++) {

```

```

63     if (s1[i] == s2[i]) {
64         matchCount++;
65         continue;
66     }
67     if (_commonDyslexicConfusions.containsKey(s1[i]) &&
68         _commonDyslexicConfusions[s1[i]]!.contains(s2[i])) {
69         matchCount++;
70     }
71 }
72 return matchCount / s1.length;
73 }

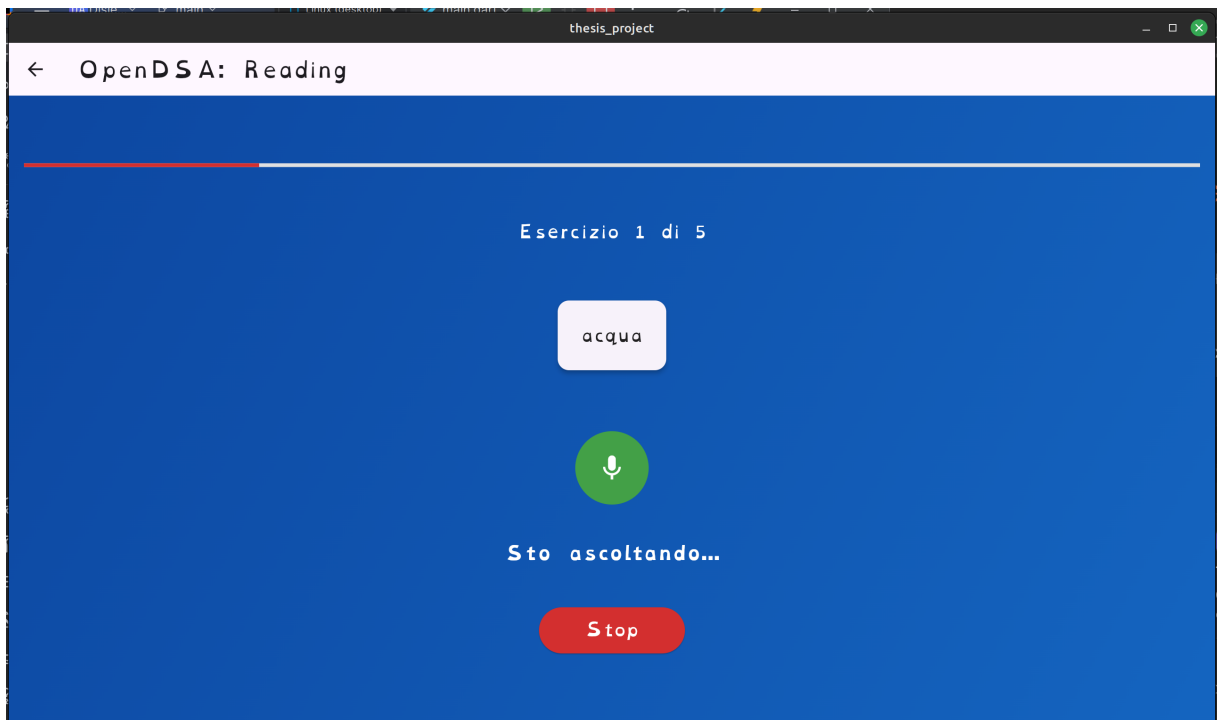
```

Listing 4.2: Calcolo della Similarita' Fonetica

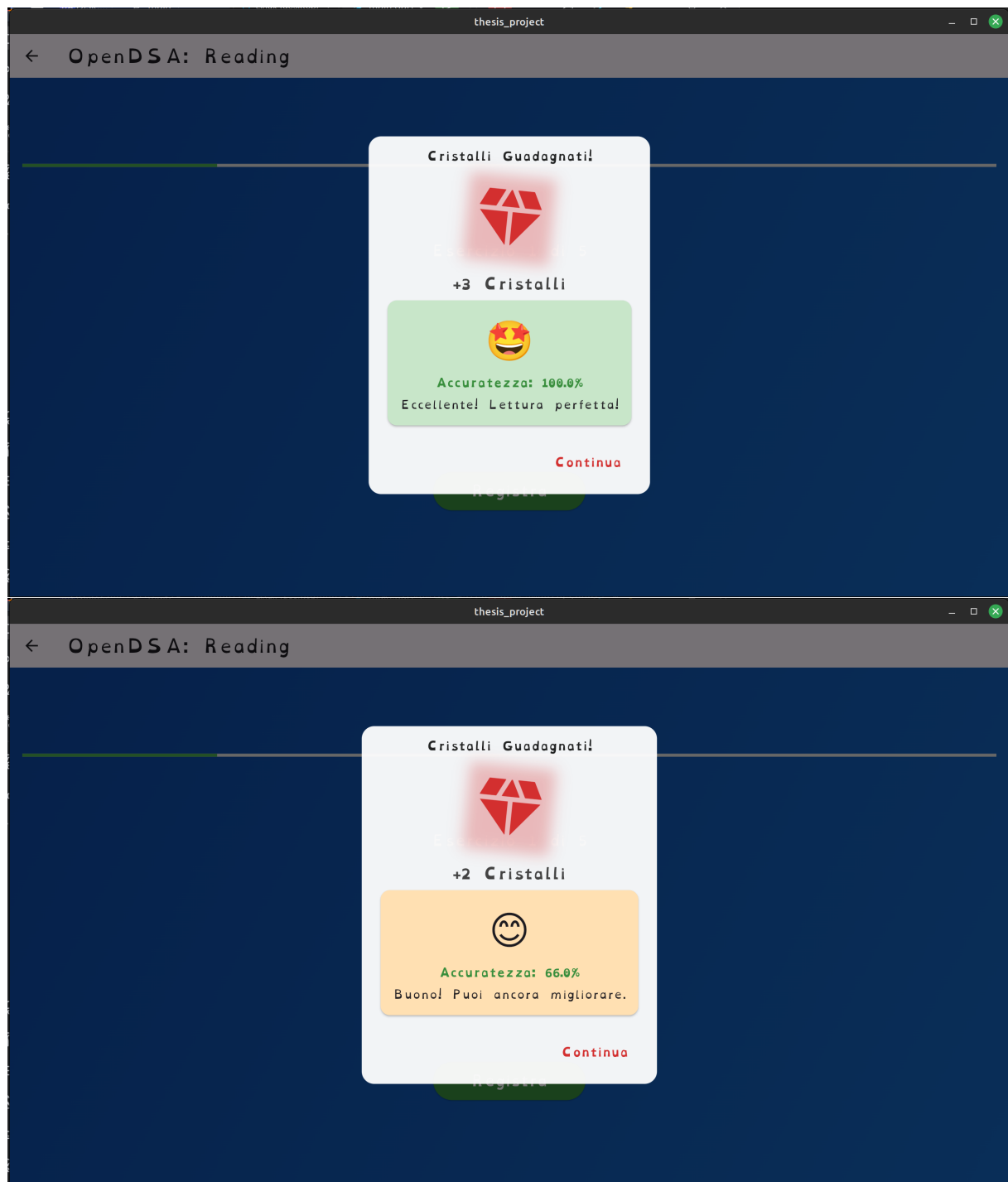
Questo ha permesso una maggiore facilità per il Modello di IA nell'interpretare correttamente quelle parole difficili per i bambini che soffrono di dislessia, avendo cura della loro difficoltà e delle eventuali problematiche che possono incorrere nell'utilizzo della App.

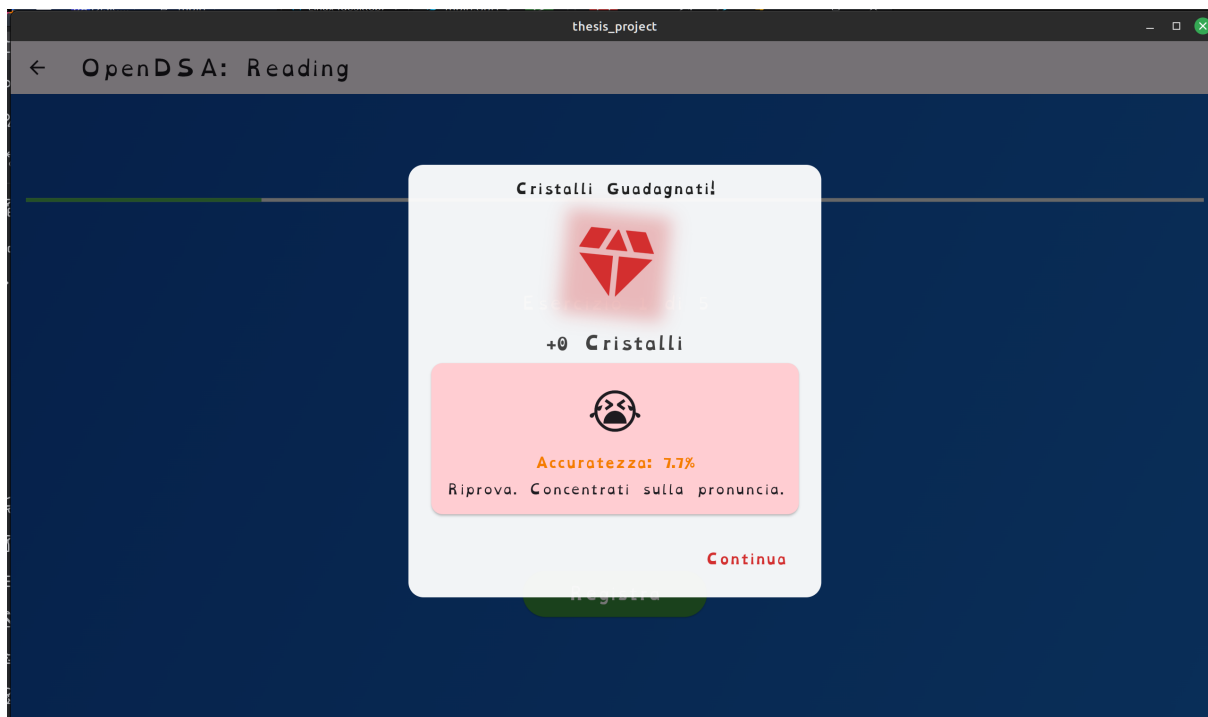
4.3.9 Il Feedback dell'Esercizio

Tornando invece alla User Journey, il passaggio successivo a quello del Riconoscimento Vocale, a seguito della produzione ed eliminazione del file audio registrato in formato ".wav" con il plugin **recorder** è il feedback utente.

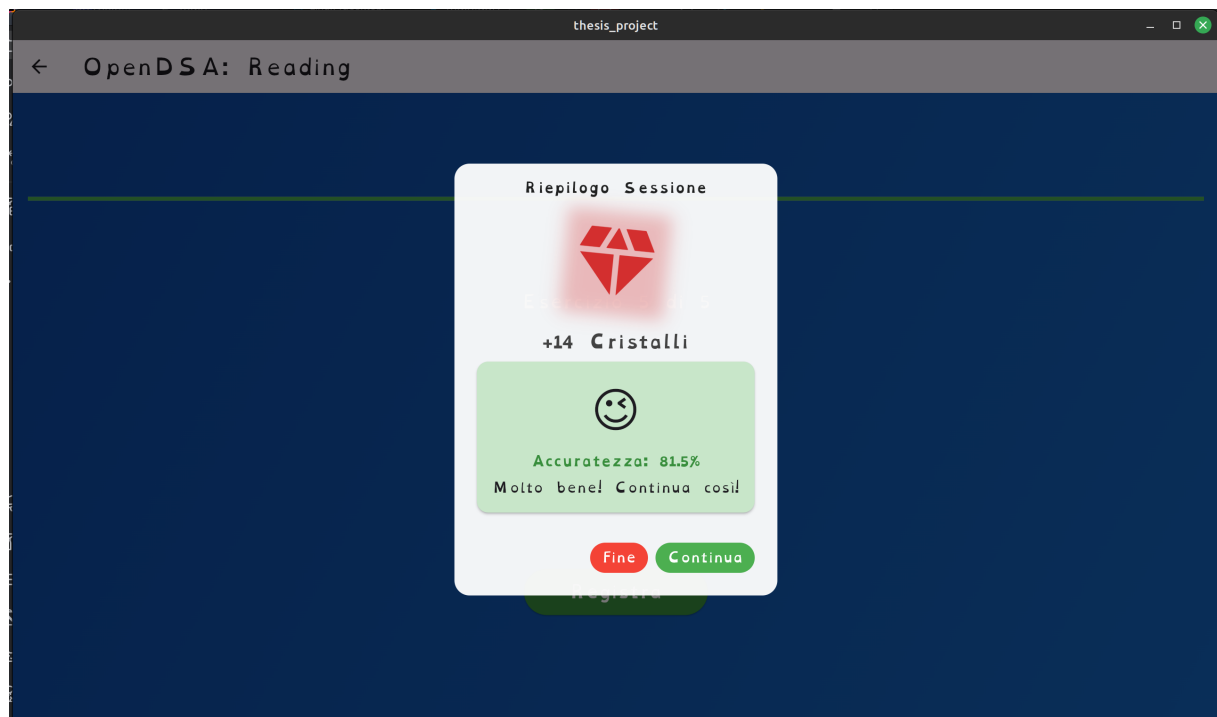


A questo proposito è bene specificare che l'output del feedback visivo è stato sapientemente pensato con colori ed emoji (caricate direttamente dal sistema essendo in codice UNICODE) in modo da dare anche un contesto ben chiaro dell'andamento dell'esercizio (oltre al riepilogo di Cristalli e Accuratezza ottenuti). Per essere ancora più meticolosi, alla fine della serie di 5 esercizi, viene proposto un quadro riepilogativo finale.





Il Gioco a questo punto ci chiederà se vogliamo Continuare, con un'altra serie di 5 esercizi, o se vogliamo concludere la nostra sessione di allenamento, tornando di fatto alla schermata principale.



4.4 Il Profilo Admin

Per il testing approfondito della App, era impensabile di poter passare 200 giorni a giocare, pertanto è stato inserito un profilo capace di "barare" per velocizzare considerevolmente il prosieguo del gioco. Tale è lo scopo del Profilo Admin.

Esso infatti parte con un quantitativo spropositato di Cristalli e la facoltà di avanzare di livello e New Game Plus pur senza fare alcun esercizio. So che non è molto "sportivo" l'implementazione di una simile scorciatoia, ma considerata la longevità garantita del Gioco, era l'unica strada percorribile per poter testare in breve tempo tutte le funzionalità del Gioco senza intoppi.

4.5 Possibili Implementazioni Future

Poichè questo è un Progetto dinamico, è stato pensato per essere fin da subito espandibile con nuovi tipi di esercizi: paragrafi, pagine e capitoli. I Primi due in realtà sono già stati impostati all'interno del codice, ma non implementati. Questo poiché risulta molto improbabile che un bambino che si avvicina a questa applicazione, voglia leggere paragrafi e pagine per il solo gioco. Pertanto sebbene l'idea sia valida ed implementabile, si è deciso di non introdurre queste tipologie di esercizio per ora, ma eventualmente in futuro con una piccola modifica del codice al link di GitHub e con l'upload contestuale di un dataset adeguato.

Anche l'introduzione di Achievements potrebbe essere una buona idea per rinforzare la struttura di Gamification e rendere l'esperienza di gioco appagante, con obiettivi secondari e badge personalizzati che sbloccano carinerie grafiche o titoli unici (o segreti) oltre alla ricompensa in cristalli.

Ovviamente, essendo Open Source, il codice può essere arricchito anche da idee fresche di collaboratori esterni che vogliono semplicemente espanderne le possibilità, lasciando il progetto resta aperto ad eventuali nuove frontiere ed espansioni.

Capitolo 5

Problemi riscontrati e Soluzioni Proposte

5.1 Creare gli Eseguibili con Flutter

Uno degli scogli che ho riscontrato nella stesura del Progetto, è stato certamente quello di creare gli eseguibili in Flutter. Di fatti, sebbene l'idea iniziale fosse quella di creare gli eseguibili per tutti i sistemi operativi direttamente dal mio computer (che usa solo Linux Mint Debian Edition), mi sono immediatamente dovuto scontrare con il limite tecnico della tecnologia, che non ti consente di creare eseguibili per Sistemi Operativi diversi da quelli in uso. Per questa ragione si è deciso, come "work around", di creare uno script shell (.sh o .bat per Windows) che consentisse la creazione degli eseguibili su macchina, una volta scaricato il progetto sul computer. Questo consente di demandare all'utente finale di scaricare eventuali dipendenze mancanti o di crearsi da sè il proprio eseguibile della app.

5.2 Apple e le sue Politiche di Distribuzione del Software

Poiché prima di oggi non avevo mai lavorato con i Sistemi di casa Cupertino, ho scoperto, mio malgrado, che per poter effettivamente creare l'eseguibile e testarlo in ambiente Apple, era necessario un Account Developer del costo di 99\$ l'anno. Poiché ritenuto non necessario, sebbene abbia implementato il supporto alla piattaforma, non ho ancora testato la App in questo ambiente.

5.3 La Scelta di Recorder come Plugin di Registrazione

Durante l'ultimazione degli ultimi test, che hanno portato poi alla stesura di questo documento, mi sono accorto che `flutter_sound` non consentiva di registrare in ambienti desktop (diversi da Linux - in quanto si usava un diverso approccio). Questo ha portato alla ricerca di un plugin che fosse integrato e plug and play per tutti gli ambienti. In Flutter, l'unico ad avere questo requisito specifico è `record`. Pertanto poco prima della fine della stesura di questo documento, ho convertito l'intero sistema di riconoscimento a `record` anziché il precedente plugin nativo di Flutter.

5.4 I Problemi di un Ambiente non Trattato per l'Audio

Poiché ho sempre avuto timore che l'ambiente d'uso in cui si potesse utilizzare la app potesse essere non trattato (magari c'è la televisione accesa o rumori di fondo) ho pensato ad un sistema di "compression" audio per la riduzione del rumore. In modo tale da eliminare possibili sbavature nella registrazione. Tuttavia, come effetto collaterale bisogna avvicinarsi di più al microfono per poterlo utilizzare agevolmente.

Capitolo 6

Conclusioni

La Creazione e lo Sviluppo di questo Progetto si è rivelata un'esperienza davvero entusiasmante e colma di sorprese, che mi ha concesso di vedere non solo uno scorcio delle potenzialità dell'Intelligenza Artificiale Generativa, ma anche quella di creare da zero una intera applicazione Stand-Alone per Computer e Mobile, attraversando tutte le fasi di Sviluppo di una Applicazione Software e toccando di fatti la maggior parte delle Materie del mio percorso di Studi, dal Primo al Terzo anno.

Sono certo che questa Applicazione sarà un'ottima aggiunta alla Suite di Ateneo che intende promuoversi per aiutare i bambini che soffrono di DSA e permetterà loro di affrontare con più gioia e serenità la loro condizione, trasformandola in un momento di sfida con se stessi e di miglioramento personale attraverso questa esperienza dedicata.

Sono certo che con il tempo arricchirò (e magari non solo io) questa applicazione, che intendo mantenere nel tempo nel mio Repository GitHub, come Progetto e come missione.

Capitolo 7

Appendice A - Elementi di Gioco

7.1 Sfide

```
1 Streak Maestro
2
3     ID: daily_streak
4     Descrizione: Mantieni una streak di 5 esercizi
5     Tipo: Giornaliero (daily)
6     Obiettivo: 5 esercizi consecutivi
7     Ricompensa: _calculateDailyReward()
8     Scadenza: Domani (tomorrow)
9
10 Precisione Perfetta
11
12     ID: daily_accuracy
13     Descrizione: Completa 3 esercizi con accuratezza superiore al
14                 90%
15     Tipo: Giornaliero (daily)
16     Obiettivo: 3 esercizi con accuratezza >90%
17     Ricompensa: _calculateDailyReward()
18     Scadenza: Domani (tomorrow)
19 Allenamento Costante
20
```

```

21     ID: weekly_exercises
22     Descrizione: Completa 20 esercizi questa settimana
23     Tipo: Settimanale (weekly)
24     Obiettivo: 20 esercizi in una settimana
25     Ricompensa: _calculateWeeklyReward()
26     Scadenza: Prossima settimana (nextWeek)
27
28 Perfezione
29
30     ID: daily_perfect
31     Descrizione: Ottieni 100% di accuratezza in 3 esercizi
32     Tipo: Giornaliero (daily)
33     Obiettivo: 3 esercizi con accuratezza del 100%
34     Ricompensa: _calculateDailyReward() * 2
35     Scadenza: Domani (tomorrow)
36
37 Super Streak Settimanale
38
39     ID: weekly_streak
40     Descrizione: Raggiungi una streak di 15 esercizi
41     Tipo: Settimanale (weekly)
42     Obiettivo: 15 esercizi consecutivi in una settimana
43     Ricompensa: _calculateWeeklyReward()
44     Scadenza: Prossima settimana (nextWeek)

```

Listing 7.1: Elenco Sfide

7.2 Trofei

```
1  Lettore Amatore
2      ID: amateur_reader
3      Nome: Trofeo del Lettore Amatore
4      Descrizione: Il primo passo nel tuo viaggio di lettura
5      Costo Base: 500
6      Icona: Icons.auto_stories
7      Colore: Colors.amber
8      Rarita': Comune
9      Numero di sequenza: 1
10
11 Lettore Allenato
12     ID: trained_reader
13     Nome: Trofeo del Lettore Allenato
14     Descrizione: Stai sviluppando le tue abilita' di lettura
15     Costo Base: 600
16     Icona: Icons.menu_book
17     Colore: Colors.green
18     Rarita': Comune
19     Numero di sequenza: 2
20
21 Lettore Principiante
22     ID: beginner_reader
23     Nome: Trofeo del Lettore Principiante
24     Descrizione: Le tue capacita' di lettura stanno crescendo
25     Costo Base: 700
26     Icona: Icons.book
27     Colore: Colors.blue
28     Rarita': Non comune
29     Numero di sequenza: 3
30
31 Bravo Lettore
32     ID: good_reader
33     Nome: Trofeo del Bravo Lettore
34     Descrizione: Stai diventando davvero bravo nella lettura
35     Costo Base: 800
36     Icona: Icons.stars
37     Colore: Colors.purple
38     Rarita': Non comune
39     Numero di sequenza: 4
40
41 Ottimo Lettore
42     ID: great_reader
```

```

43     Nome: Trofeo dell'Ottimo Lettore
44     Descrizione: Le tue capacita' di lettura sono eccellenti
45     Costo Base: 900
46     Icona: Icons.workspace_premium
47     Colore: Colors.orange
48     Rarita': Raro
49     Numero di sequenza: 5
50
51     Super Lettore
52     ID: super_reader
53     Nome: Trofeo del Super Lettore
54     Descrizione: Hai raggiunto un livello superiore di lettura
55     Costo Base: 1000
56     Icona: Icons.psychology
57     Colore: Colors.red
58     Rarita': Raro
59     Numero di sequenza: 6
60
61     Gran Lettore
62     ID: grand_reader
63     Nome: Trofeo del Gran Lettore
64     Descrizione: Sei diventato un lettore eccezionale
65     Costo Base: 1200
66     Icona: Icons.grade
67     Colore: Colors.indigo
68     Rarita': Epico
69     Numero di sequenza: 7
70
71     Lettore Maestro
72     ID: master_reader
73     Nome: Trofeo del Lettore Maestro
74     Descrizione: Hai padroneggiato l'arte della lettura
75     Costo Base: 1500
76     Icona: Icons.emoji_events
77     Colore: Colors.deepPurple
78     Rarita': Epico
79     Numero di sequenza: 8
80
81     Gran Maestro Lettore
82     ID: grandmaster_reader
83     Nome: Trofeo del Gran Maestro Lettore
84     Descrizione: Il piu' alto riconoscimento per un lettore
85     Costo Base: 2000
86     Icona: Icons.military_tech

```



```
87     Colore: Colors.teal
88     Rarita': Leggendario
89     Numero di sequenza: 9
90
91     Gran Maestro Supremo Della Lettura
92
93         ID: supreme_grandmaster_reader
94         Nome: Trofeo del Gran Maestro Supremo Della Lettura
95         Descrizione: La Consacrazione nella Leggenda per un
96                     lettore
97         Costo Base: 1.000.000
98         Icona: Icons.stadium_rounded
99         Colore: Colors.amber.shade800
100        Rarita': Leggendario
        Numero di sequenza: 10
```

Listing 7.2: Elenco Trofei

7.3 Livelli

```
1 Livello 1
2
3     Numero: 1
4     Cristallo: Rosso (CrystalType.Red)
5     Parole target: 30
6     Sotto-livello:
7         Nome: Parole Semplici
8         Lunghezza parole: 3-5 caratteri
9     Moltiplicatori di difficolta': defaultMultipliers
10
11 Livello 2
12
13     Numero: 2
14     Cristallo: Arancione (CrystalType.Orange)
15     Parole target: 20
16     Sotto-livello:
17         Nome: Parole Medie
18         Lunghezza parole: 6-8 caratteri
19     Moltiplicatori di difficolta': defaultMultipliers
20
21 Livello 3
22
23     Numero: 3
24     Cristallo: Giallo (CrystalType.Yellow)
25     Parole target: 20
26     Sotto-livello:
27         Nome: Parole Difficili
28         Lunghezza parole: 9-12 caratteri
29     Moltiplicatori di difficolta': defaultMultipliers
30
31 Livello 4
32
33     Numero: 4
34     Cristallo: Verde (CrystalType.Green)
35     Parole target: 15
36     Sotto-livello:
37         Nome: Frasi
38         Lunghezza frase: 1 frase
39     Moltiplicatori di difficolta':
40         Facile: 1.0
41         Medio: 1.8
42         Difficile: 2.5
```

Listing 7.3: Elenco Livelli

7.4 Configurazioni dell'Applicazione

Parametro	Tipo	Descrizione
appName	Stringa	Nome dell'applicazione
appVersion	Stringa	Versione corrente dell'app
Configurazioni Font		
title	Double	Dimensione del font per i titoli
subtitle	Double	Dimensione del font per i sottotitoli
others	Double	Dimensione del font per altri testi
Configurazioni VOSK (Riconoscimento Vocale)		
voskModelUrl	Stringa	URL del modello VOSK per il riconoscimento vocale
voskModelName	Stringa	Nome del modello VOSK
voskModelVersion	Stringa	Versione del modello VOSK
vadAggressiveness	Int	Livello di aggressività del VAD (0-3)
vadThreshold	Double	Soglia per il Voice Activity Detection
vadEnable	Booleano	Attiva/disattiva il VAD
noiseSuppressionEnable	Booleano	Attiva/disattiva la soppressione del rumore
autoGainControlEnable	Booleano	Attiva/disattiva il controllo automatico del guadagno
Configurazioni Audio		
sampleRate	Int	Frequenza di campionamento audio (Hz)
<i>Continua nella pagina successiva</i>		

Continua dalla pagina precedente

Parametro	Tipo	Descrizione
channels	Int	Numero di canali audio
bufferSize	Int	Dimensione del buffer audio
volumeThreshold	Double	Soglia minima di volume per considerare un input valido
idealVolume	Double	Volume ideale per una registrazione ottimale
maxVolume	Double	Volume massimo accettabile
Configurazioni Riconoscimento		
minSimilarityScore	Double	Punteggio minimo di similarità richiesto
perfectSimilarityScore	Double	Punteggio perfetto di similarità
maxRecordingDuration	Int	Durata massima di una registrazione (secondi)
minRecordingDuration	Int	Durata minima di una registrazione (secondi)
Configurazioni Game		
basePointsWord	Int	Punti base per parola corretta
basePointsSentence	Int	Punti base per frase corretta
basePointsParagraph	Int	Punti base per paragrafo corretto
basePointsPage	Int	Punti base per pagina corretta
Configurazioni Feedback		
<i>Continua nella pagina successiva</i>		

Continua dalla pagina precedente

Parametro	Tipo	Descrizione
defaultVibrationEnabled	Booleano	Attiva/disattiva la vibrazione
defaultSoundEnabled	Booleano	Attiva/disattiva il feedback sonoro
defaultVisualEnabled	Booleano	Attiva/disattiva il feedback visivo
Configurazioni Cache		
maxCacheSize	Int	Dimensione massima della cache (byte)
cacheExpiration	Durata	Tempo di scadenza della cache
Configurazioni Learning Analytics		
maxStoredSessions	Int	Numero massimo di sessioni salvate
minSessionDuration	Int	Durata minima di una sessione (secondi)
maxSessionDuration	Int	Durata massima di una sessione (secondi)
Configurazioni UI		
animationDuration	Durata	Durata delle animazioni UI
defaultFontSize	Double	Dimensione del font predefinita
headerFontSize	Double	Dimensione del font per gli header
buttonHeight	Double	Altezza standard dei pulsanti
cardElevation	Double	Elevazione standard delle card
<i>Continua nella pagina successiva</i>		

Continua dalla pagina precedente

Parametro	Tipo	Descrizione
Supporto per New Game+		
<code>newGamePlusMultipliers</code>	Mappa	Moltiplicatori di punteggio per i vari livelli di New Game+
Path delle Risorse		
<code>wordsEasyPath</code>	Stringa	Percorso file per parole facili
<code>wordsMediumPath</code>	Stringa	Percorso file per parole medie
<code>wordsHardPath</code>	Stringa	Percorso file per parole difficili
<code>sentencesPath</code>	Stringa	Percorso file per le frasi
<code>paragraphsPath</code>	Stringa	Percorso file per i paragrafi
<code>pagesPath</code>	Stringa	Percorso file per le pagine

Capitolo 8

Appendice B - Codice

Qui di seguito si riportano i codici delle Classi Principali che Consentono all'App di riconoscere l'audio che viene registrato:

8.1 VoskService

```
1 import 'dart:async';
2 import 'dart:io';
3 import 'dart:convert';
4 import 'dart:typed_data';
5 import 'dart:math' show min, max;
6 import 'package:flutter/material.dart';
7 import 'package:flutter/services.dart';
8 import 'package:path/path.dart' as path;
9 import 'package:path_provider/path_provider.dart';
10 import 'package:vosk_flutter/vosk_flutter.dart';
11 import '../models/recognition_result.dart';
12 import '../config/app_config.dart';
13 import '../utils/permission_handler.dart';
14 import 'permission_service.dart';
15 import 'audio_service.dart';
16 import '../utils/desktop_permission.dart';
17
18
19 /// Servizio per il riconoscimento vocale utilizzando VOSK.
```

```

20 /// Legge il file WAV (con header) e passa direttamente il buffer
    dei byte
21 /// al recognizer tramite acceptWaveformBytes(), per poi ottenere il
    risultato
22 /// con getFinalResult().
23 class VoskService {
24     static final VoskService _instance = VoskService._internal();
25     static VoskService get instance => _instance;
26
27     VoskFlutterPlugin? _recognizer;
28     Model? _model;
29     Recognizer? _speechRecognizer;
30     SpeechService? _speechService;
31
32     final PermissionService _permissionService = PermissionService();
33     final AudioService _audioService = AudioService();
34
35     bool _isInitialized = false;
36     String _modelPath = '';
37     bool _isProcessing = false;
38
39     static const List<String> _requiredFiles = [
40         'am/final.mdl',
41         'conf/mfcc.conf',
42         'conf/model.conf',
43         'graph/Gr.fst',
44         'graph/HCLr.fst',
45         'graph/disambig_tid.int',
46         'graph/phones/word_boundary.int',
47         'ivector/final.dubm',
48         'ivector/final.ie',
49         'ivector/final.mat',
50         'ivector/global_cmvn.stats',
51         'ivector/online_cmvn.conf',
52         'ivector/splice.conf',
53     ];
54
55     final List<String> _serviceLog = [];
56     static const int _maxInitAttempts = 3;
57
58     // Mappa per errori comuni (usata nelle funzioni di similarita')
59     static const Map<String, List<String>> _commonDyslexicConfusions =
        {
60         'b': ['d', 'p'],

```

```

61     'd': ['b', 'q'],
62     'p': ['q', 'b'],
63     'q': ['p', 'd'],
64     'm': ['n', 'w'],
65     'n': ['m'],
66     'a': ['e'],
67     'e': ['a'],
68     's': ['z'],
69     'z': ['s'],
70     'f': ['v'],
71     'v': ['f'],
72 };
73
74 VoskService._internal() {
75     _logEvent('VoskService inizializzato');
76     _initAudioService();
77 }
78
79 /// Inizializza il servizio audio.
80 void _initAudioService() {
81     if (!_audioService.isInitialized) {
82         _audioService.initialize();
83     }
84 }
85
86 /// Registra un evento con timestamp nei log.
87 void _logEvent(String event) {
88     final timestamp = DateTime.now().toIso8601String();
89     final logEntry = '[$timestamp] $event';
90     debugPrint('VoskService: $logEntry');
91     _serviceLog.add(logEntry);
92     if (_serviceLog.length > 1000) _serviceLog.removeAt(0);
93 }
94
95 /// Inizializza il servizio VOSK e prepara il modello.
96 Future<void> initialize({required BuildContext context}) async {
97     if (!_isInitialized) {
98         _logEvent('Servizio gia' inizializzato');
99         return;
100     }
101     int attempts = 0;
102     bool success = false;
103     while (!success && attempts < _maxInitAttempts) {
104         try {

```

```

105         attempts++;
106         _logEvent('Tentativo di inizializzazione #$attempts');
107         await _initializeWithRetry(context: context);
108         success = true;
109     } catch (e, stackTrace) {
110         _logEvent('Errore nel tentativo #$attempts: $e');
111         _logEvent('Stack trace: $stackTrace');
112         if (attempts >= _maxInitAttempts) {
113             throw Exception('Impossibile inizializzare VOSK dopo
114                 $_maxInitAttempts tentativi');
115         }
116         await Future.delayed(const Duration(seconds: 1));
117     }
118 }
119
120 Future<void> _initializeWithRetry({required BuildContext context})
121     async {
122         _logEvent('Inizio inizializzazione con retry');
123
124         // Verifica permessi su mobile o desktop
125         if (Platform.isAndroid || Platform.isIOS) {
126             // Prima richiedi il permesso di storage, poi quello del
127             // microfono
128             bool hasStoragePermission = await PermissionsHandler.
129                 requestStoragePermission();
130             if (!hasStoragePermission) {
131                 _logEvent('ERRORE: Permesso di storage non concesso');
132                 throw Exception('Permesso di storage non concesso');
133             }
134
135             bool hasMicrophonePermission = await PermissionsHandler.
136                 requestMicrophonePermission();
137             if (!hasMicrophonePermission) {
138                 _logEvent('ERRORE: Permesso del microfono non concesso');
139                 throw Exception('Permesso del microfono non concesso');
140             }
141
142             _logEvent('Permessi concessi: Storage e Microfono');
143         } else {
144             // Su desktop
145             final hasAudioAccess = await DesktopPermission.
146                 checkMicrophoneAccess();

```

```

142     final hasStorageAccess = await DesktopPermission.
        requestStorageAccess();
143
144     if (!hasAudioAccess) {
145         _logEvent('ERRORE: Permesso audio non disponibile su Desktop
        ');
146         throw Exception('Permesso audio non disponibile su Desktop')
        ;
147     }
148
149     if (!hasStorageAccess) {
150         _logEvent('ERRORE: Permesso storage non disponibile su
        Desktop');
151         throw Exception('Permesso storage non disponibile su Desktop
        ');
152     }
153
154     _logEvent('Permessi desktop verificati: Audio=$hasAudioAccess,
        Storage=$hasStorageAccess');
155 }
156
157 // Trova il percorso del modello e verifica la sua integrita'.
158 _modelPath = await _findModelPath();
159 _logEvent('Percorso del modello impostato: $_modelPath');
160 if (!await _verifyModelIntegrity(_modelPath)) {
161     throw Exception('Integrita' del modello non verificata in
        $_modelPath');
162 }
163
164 _logEvent('Inizializzazione componenti VOSK');
165 _recognizer = VoskFlutterPlugin.instance();
166 _model = await _recognizer!.createModel(_modelPath);
167 _speechRecognizer = await _recognizer!.createRecognizer(
168     model: _model!,
169     sampleRate: AppConfig.sampleRate,
170 );
171
172 if (Platform.isAndroid || Platform.isIOS) {
173     _logEvent('Initializing SpeechService (mobile)...');
174     _speechService = await _recognizer!.initSpeechService(
        _speechRecognizer!);
175 } else {
176     _logEvent("Saltata initSpeechService su Desktop platforms.");
177     _speechService = null;

```

```

178     }
179
180     if (_speechRecognizer != null) {
181         await _speechRecognizer!.setMaxAlternatives(3);
182         await _speechRecognizer!.setPartialWords(partialWords: true);
183         await _speechRecognizer!.setWords(words: true);
184     }
185
186     _isInitialized = true;
187     _logEvent('Inizializzazione completata con successo');
188 }
189
190 /// Trova il percorso del modello VOSK.
191 Future<String> _findModelPath() async {
192     try {
193         if (Platform.environment.containsKey('FLUTTER_TEST')) {
194             return path.join(Directory.current.path, 'assets', 'vosk');
195         }
196         final appDocDir = await getApplicationDocumentsDirectory();
197         final modelDirPath = path.join(appDocDir.path, 'vosk');
198         final modelDir = Directory(modelDirPath);
199         if (!await modelDir.exists()) {
200             _logEvent('Cartella modello non esistente. Creazione e copia
                degli asset...');
201             await modelDir.create(recursive: true);
202             for (final assetFile in _requiredFiles) {
203                 final assetPath = path.join('vosk', assetFile);
204                 try {
205                     final byteData = await rootBundle.load(assetPath);
206                     final targetFile = File(path.join(modelDirPath,
                        assetFile));
207                     await targetFile.parent.create(recursive: true);
208                     await targetFile.writeAsBytes(byteData.buffer.
                        asUint8List());
209                     _logEvent('Asset copiato: $assetFile');
210                 } catch (e) {
211                     _logEvent('Errore nella copia dell\'asset $assetFile: $e
                        ');
212                 }
213             }
214         } else {
215             _logEvent('Cartella modello gia\' esistente: $modelDirPath');
216         }
217         return modelDirPath;

```

```

218     } catch (e) {
219         _logEvent('Errore nella ricerca del modello: $e');
220         rethrow;
221     }
222 }
223
224 /// Verifica l'integrita' del modello controllando la presenza di
tutti i file richiesti.
225 Future<bool> _verifyModelIntegrity(String modelPath) async {
226     debugPrint('VoskService: Verifica integrita' modello in:
227         $modelPath');
228     try {
229         for (final file in _requiredFiles) {
230             final fullPath = path.join(modelPath, file);
231             if (!await File(fullPath).exists()) {
232                 debugPrint('VoskService: File mancante: $file');
233                 return false;
234             }
235             debugPrint('VoskService: File presente: $file');
236         }
237         debugPrint('VoskService: Verifica integrita' completata con
238             successo');
239         return true;
240     } catch (e) {
241         debugPrint('VoskService: Errore nella verifica integrita': $e'
242             );
243         return false;
244     }
245 }
246
247 /// Avvia il riconoscimento vocale reale.
248 /// Legge il file WAV (con header) e passa direttamente il buffer
dei byte al recognizer.
249 Future<RecognitionResult> startRecognition(String targetText, [
250     String? audioPath]) async {
251     _logEvent('Avvio riconoscimento vocale per target: $targetText')
252     ;
253     if (!_isInitialized) {
254         throw Exception('VoskService non inizializzato');
255     }
256     if (_isProcessing) {
257         throw Exception('Processo di riconoscimento gia' in corso');
258     }
259     final startTime = DateTime.now();

```

```

255     _isProcessing = true;
256     try {
257         String pathToProcess = audioPath ?? "";
258         if (pathToProcess.isEmpty) {
259             // Avvia la registrazione tramite AudioService.
260             pathToProcess = await _audioService.startRecording()
261                 .timeout(const Duration(seconds: 10), onTimeout: () {
262                     _logEvent('Timeout nell\'avvio della registrazione');
263                     throw Exception('Timeout nell\'avvio della registrazione')
264                 });
265             if (pathToProcess.isEmpty) {
266                 _logEvent('Percorso registrazione vuoto');
267                 throw Exception('Registrazione audio fallita');
268             }
269             await Future.delayed(const Duration(seconds: 3))
270                 .timeout(const Duration(seconds: 10), onTimeout: () {
271                     _logEvent('Timeout nella durata della registrazione');
272                     throw Exception('Timeout nella durata della registrazione')
273                 });
274             await _audioService.stopRecording()
275                 .timeout(const Duration(seconds: 5), onTimeout: () {
276                     _logEvent('Timeout nello stop della registrazione');
277                     throw Exception('Timeout nello stop della registrazione');
278                 });
279         }
280
281         _logEvent('Lettura file audio da: $pathToProcess');
282         final bytes = await File(pathToProcess).readAsBytes();
283         if (bytes.isEmpty) {
284             _logEvent('Nessun dato audio valido');
285             throw Exception('Nessun dato audio valido');
286         }
287
288         _logEvent('Invio a VOSK dei byte audio (${bytes.length} byte)
289             in una singola chiamata');
290         await _speechRecognizer!.acceptWaveformBytes(bytes)
291             .timeout(const Duration(seconds: AppConfig.
292                 maxRecordingDuration), onTimeout: () {
293                 _logEvent('Timeout nell\'accettazione dei byte del waveform'
294                     );
295                 throw Exception('Timeout nell\'accettazione dei byte del
296                     waveform');

```



```

293     });
294
295     // Richiede direttamente il risultato finale.
296     final resultJson = await _speechRecognizer!.getFinalResult()
297         .timeout(const Duration(seconds: 5), onTimeout: () {
298         _logEvent('Timeout nell\'ottenimento del risultato finale');
299         throw Exception('Timeout nel risultato finale');
300     });
301
302     _logEvent('Elaborazione VOSK completata');
303     _logEvent('Risultato raw da VOSK: $resultJson');
304
305     final cleanJson = _cleanJsonFormat(resultJson);
306     Map<String, dynamic> jsonResult;
307     try {
308         jsonResult = jsonDecode(cleanJson);
309     } catch (jsonError) {
310         debugPrint('VoskService: Errore nel parsing JSON: $jsonError');
311         throw Exception('Errore nel parsing JSON: $jsonError');
312     }
313
314     double totalConfidence = 0.0;
315     String recognizedText = '';
316
317     // Gestione del caso in cui il JSON contiene "alternatives"
318     if (jsonResult.containsKey('alternatives')) {
319         final List<dynamic> alternatives = jsonResult['alternatives'];
320
321         if (alternatives.isNotEmpty) {
322             final alt = alternatives[0];
323             if (alt is Map && alt.containsKey('text')) {
324                 recognizedText = (alt['text'] as String?).trim() ?? '';
325             }
326             if (alt is Map && alt.containsKey('confidence')) {
327                 try {
328                     var conf = alt['confidence'];
329                     if (conf is num) {
330                         totalConfidence = conf.toDouble();
331                     } else if (conf is String) {
332                         totalConfidence = double.tryParse(conf.replaceAll(
333                             ', ', '.')) ?? 0.0;
334                     }
335                 } catch (e) {

```

```

334         totalConfidence = 0.2;
335     }
336 }
337 }
338 }
339 // Se non ho ottenuto un risultato da "alternatives",
    controllo "result" oppure "text"
340 else if (jsonResult.containsKey('result')) {
341     final List<dynamic> words = jsonResult['result'];
342     for (var word in words) {
343         if (word is Map && word.containsKey('word')) {
344             recognizedText += '${word['word']}' ';
345             if (word.containsKey('conf')) {
346                 try {
347                     var conf = word['conf'];
348                     if (conf is num) {
349                         totalConfidence += conf.toDouble();
350                     } else if (conf is String) {
351                         totalConfidence += double.tryParse(conf.replaceAll(
352                             ',', '.', '')) ?? 0.0;
353                     }
354                 } catch (e) {
355                     debugPrint('VoskService: Errore nell'estrazione
356                         della confidenza: $e');
357                 }
358             }
359         }
360     }
361     recognizedText = recognizedText.trim();
362     totalConfidence = words.isEmpty ? 0.0 : totalConfidence /
        words.length;
363 } else if (jsonResult.containsKey('text')) {
364     recognizedText = (jsonResult['text'] as String)?.trim() ??
        '';
365     totalConfidence = 0.2;
366 } else {
367     recognizedText = '';
368     totalConfidence = 0.0;
369 }
370 if (totalConfidence > 1.0) {
371     totalConfidence = totalConfidence / 100.0;
372 }
373 if (totalConfidence <= 0) {

```

```

373         totalConfidence = 0.1;
374     }
375
376     if (recognizedText.isEmpty) {
377         _logEvent('Nessun testo riconosciuto da VOSK');
378         throw Exception('Riconoscimento fallito: nessun testo
379             riconosciuto');
380     }
381
382     final similarity = _calculateTextSimilarity(
383         recognizedText.toLowerCase(), targetText.toLowerCase());
384     final isCorrect = similarity >= AppConfig.minSimilarityScore;
385
386     return RecognitionResult(
387         text: recognizedText,
388         confidence: totalConfidence,
389         similarity: similarity,
390         isCorrect: isCorrect,
391         duration: DateTime.now().difference(startTime),
392     );
393 } catch (e) {
394     _logEvent('Errore nel riconoscimento: $e');
395     rethrow;
396 } finally {
397     _isProcessing = false;
398 }
399
400 String _cleanJsonFormat(String jsonString) {
401     var result = jsonString;
402     result = result.replaceAllMapped(RegExp(r'(\d+),(\d+)'), (match)
403         {
404         final parts = match.group(0)!.split(',');
405         return "${parts[0]}.${parts[1]}";
406     });
407     result = result.replaceAll("'", '"');
408     result = result.replaceAll("NaN", "0");
409     result = result.replaceAll("Infinity", "999999");
410     result = result.replaceAll(RegExp(r'[\u0000-\u001F]'), '');
411     return result;
412 }
413
414 double _calculateTextSimilarity(String text1, String text2) {
415     if (text1 == text2) return 1.0;

```

```

415     if (text1.isEmpty || text2.isEmpty) return 0.0;
416     const double levenshteinWeight = 0.4;
417     const double phoneticWeight = 0.3;
418     const double confusionWeight = 0.3;
419     double levenshteinScore = _calculateLevenshteinSimilarity(text1,
420         text2);
421     double phoneticScore = _calculatePhoneticSimilarity(text1, text2
422         );
423     double confusionScore = _calculateConfusionSimilarity(text1,
424         text2);
425     return (levenshteinScore * levenshteinWeight) +
426         (phoneticScore * phoneticWeight) +
427         (confusionScore * confusionWeight);
428 }
429
430 double _calculateConfusionSimilarity(String s1, String s2) {
431     if (s1.length != s2.length) return 0.0;
432     int matchCount = 0;
433     for (int i = 0; i < s1.length; i++) {
434         if (s1[i] == s2[i]) {
435             matchCount++;
436             continue;
437         }
438         if (_commonDyslexicConfusions.containsKey(s1[i]) &&
439             _commonDyslexicConfusions[s1[i]]!.contains(s2[i])) {
440             matchCount++;
441         }
442     }
443     return matchCount / s1.length;
444 }
445
446 double _calculateLevenshteinSimilarity(String s1, String s2) {
447     var matrix = List.generate(
448         s1.length + 1,
449         (i) => List.generate(s2.length + 1, (j) => j == 0 ? i : 0)
450         ,
451     );
452     for (var j = 0; j <= s2.length; j++) {
453         matrix[0][j] = j;
454     }
455     for (var i = 1; i <= s1.length; i++) {
456         for (var j = 1; j <= s2.length; j++) {
457             int cost = _calculateSubstitutionCost(s1[i - 1], s2[j - 1]);
458             matrix[i][j] = min(

```

```

455         matrix[i - 1][j] + 1,
456         min(matrix[i][j - 1] + 1, matrix[i - 1][j - 1] + cost));
457     if (i > 1 &&
458         j > 1 &&
459         s1[i - 1] == s2[j - 2] &&
460         s1[i - 2] == s2[j - 1]) {
461         matrix[i][j] = min(matrix[i][j], matrix[i - 2][j - 2] + 1)
462         ;
463     }
464 }
465 final maxLength = max(s1.length, s2.length);
466 return 1.0 - (matrix[s1.length][s2.length] / maxLength);
467 }
468
469 int _calculateSubstitutionCost(String char1, String char2) {
470     if (char1 == char2) return 0;
471     if (_commonDyslexicConfusions.containsKey(char1) &&
472         _commonDyslexicConfusions[char1]!.contains(char2)) {
473         return 1;
474     }
475     return 2;
476 }
477
478 double _calculatePhoneticSimilarity(String s1, String s2) {
479     String phonetic1 = _getPhoneticCode(s1);
480     String phonetic2 = _getPhoneticCode(s2);
481     return _calculateLevenshteinSimilarity(phonetic1, phonetic2);
482 }
483
484 String _getPhoneticCode(String text) {
485     var result = text.toLowerCase();
486     final Map<String, String> phoneticRules = {
487         'chi': 'ki',
488         'che': 'ke',
489         'ghi': 'gi',
490         'ghe': 'ge',
491         'gn': 'ñ',
492         'gl': 'Ŀ',
493         'sc': 'ſ',
494         'qu': 'k',
495         'gu': 'g',
496         'z': 'ts',
497         'zz': 'ts',

```

```

498     };
499     for (var entry in phoneticRules.entries) {
500         result = result.replaceAll(entry.key, entry.value);
501     }
502     result = result.replaceAll(RegExp(r'([aeiou])\1+'), r'$1');
503     return result;
504 }
505
506 List<String> getServiceLogs() => List.unmodifiable(_serviceLog);
507
508 bool get isInitialized => _isInitialized;
509 String get modelPath => _modelPath;
510 }

```

Listing 8.1: vosk_service.dart

8.2 SpeechRecognitionService

La Classe Orchestratore, è lei che giostra i Servizi connettendoli l'un l'altro demandando le risorse necessarie al loro funzionamento.

```

1 // lib/services/speech_recognition_service.dart
2
3 import 'dart:async';
4 import 'dart:convert';
5 import 'dart:io';
6 import 'package:flutter/material.dart';
7 import 'package:path_provider/path_provider.dart';
8 import '../models/recognition_result.dart';
9 import '../services/audio_service.dart';
10 import '../services/vosk_service.dart';
11 import '../utils/custom_linux_recorder.dart';
12
13 /// Stati possibili del servizio di riconoscimento vocale
14 enum RecognitionState {
15     initializing, // Inizializzazione in corso
16     idle, // Pronto ma non in uso
17     recording, // Registrazione attiva
18     processing, // Elaborazione audio in corso
19     completed, // Riconoscimento completato

```

```

20     error          // Errore
21 }
22
23 /// Servizio per il riconoscimento vocale che coordina la
24 registrazione audio
25 /// e il processamento tramite il modello VOSK.
26 class SpeechRecognitionService {
27     // Servizi utilizzati
28     final AudioService _audioService = AudioService();
29     final VoskService _voskService = VoskService.instance;
30     CustomLinuxRecorder? _linuxRecorder; // Per Linux
31
32     // Stream controllers
33     final _stateController = StreamController<RecognitionState>.
34         broadcast();
35     final _resultController = StreamController<RecognitionResult>.
36         broadcast();
37     final _errorController = StreamController<String>.broadcast();
38     final _volumeController = StreamController<double>.broadcast();
39
40     // Stato corrente
41     RecognitionState _currentState = RecognitionState.idle;
42     String? _currentTarget;
43     String? _recordingPath;
44     bool _isInitialized = false;
45
46     // Stream pubblici
47     Stream<RecognitionState> get stateStream => _stateController.
48         stream;
49     Stream<RecognitionResult> get resultStream => _resultController.
50         stream;
51     Stream<String> get errorStream => _errorController.stream;
52     Stream<double> get volumeStream => _volumeController.stream;
53
54     // Getters
55     RecognitionState get currentState => _currentState;
56     AudioService get audioService => _audioService;
57
58     /// Inizializza il servizio
59     Future<void> initialize(BuildContext context) async {
60         if (!_isInitialized) {
61             debugPrint('SpeechRecognitionService: Già' inizializzato');
62             return;
63         }
64     }

```

```

59
60 debugPrint('SpeechRecognitionService: Inizializzazione');
61 _updateState(RecognitionState.initializing);
62
63 try {
64     // Inizializza prima l'audio service
65     await _audioService.initialize();
66
67     // Su Linux, inizializza anche il recorder personalizzato
68     if (Platform.isLinux) {
69         _linuxRecorder = CustomLinuxRecorder();
70         await _linuxRecorder!.initialize();
71
72         // Inoltre gli eventi di volume dal recorder Linux
73         _linuxRecorder!.amplitudeStream.listen((level) {
74             _volumeController.add(level);
75         });
76     }
77
78     // Inizializza VOSK
79     await _voskService.initialize(context: context);
80
81     // Inoltre gli eventi di volume dall'audioService
82     _audioService.volumeLevel.listen((volume) {
83         _volumeController.add(volume);
84     });
85
86     _isInitialized = true;
87     _updateState(RecognitionState.idle);
88     debugPrint('SpeechRecognitionService: Inizializzazione
        completata');
89 } catch (e) {
90     _emitError('Errore nell\'inizializzazione: $e');
91     _updateState(RecognitionState.error);
92     rethrow;
93 }
94 }
95
96 /// Avvia il riconoscimento per il testo target
97 Future<void> startRecognition(String targetText) async {
98     if (_currentState == RecognitionState.recording ||
99         _currentState == RecognitionState.processing) {
100         _emitError('Riconoscimento gia' in corso');
101         return;

```



```

102     }
103
104     if (!_isInitialized) {
105         _emitError('Servizio non inizializzato');
106         return;
107     }
108
109     _currentTarget = targetText;
110     _updateState(RecognitionState.recording);
111
112     try {
113         // Generazione di un path unico basato sul timestamp
114         final timestamp = DateTime.now().millisecondsSinceEpoch;
115         final appDocDir = await getApplicationDocumentsDirectory();
116         final recordingDir = Directory('${appDocDir.path}/
            OpenDSA_recordings');
117         if (!await recordingDir.exists()) {
118             await recordingDir.create(recursive: true);
119         }
120
121         // Utilizziamo lo stesso percorso per tutte le piattaforme
122         _recordingPath = '${recordingDir.path}/recording_${timestamp}.
            wav';
123         debugPrint('SpeechRecognitionService: Percorso di
            registrazione generato: $_recordingPath');
124
125         if (Platform.isLinux && _linuxRecorder != null) {
126             // Su Linux, passa il percorso generato al recorder
            personalizzato
127             final started = await _linuxRecorder!.start(_recordingPath!)
                ;
128             if (!started) {
129                 throw Exception('Impossibile avviare la registrazione con
                    il recorder personalizzato');
130             }
131         } else {
132             // Su altre piattaforme, usa AudioService con lo stesso
            percorso
133             await _audioService.startRecording();
134             if (_recordingPath == null || _recordingPath!.isEmpty) {
135                 throw Exception('Impossibile avviare la registrazione');
136             }
137         }
138     } catch (e) {

```

```

139     _emitError('Errore nell\'avvio della registrazione: $e');
140     _updateState(RecognitionState.error);
141 }
142 }
143
144 /// Ferma la registrazione e avvia l'elaborazione
145 Future<void> stopRecognition() async {
146     if (_currentState != RecognitionState.recording) {
147         _emitError('Nessuna registrazione attiva');
148         return;
149     }
150
151     _updateState(RecognitionState.processing);
152
153     try {
154         String? actualRecordingPath = null;
155
156         if (Platform.isLinux && _linuxRecorder != null) {
157             // Su Linux, usa il recorder personalizzato
158             actualRecordingPath = await _linuxRecorder!.stop();
159             if (actualRecordingPath == null) {
160                 throw Exception('Errore nello stop della registrazione
161                     Linux');
162             }
163             // Importante: aggiorna il percorso di registrazione con
164             quello effettivamente usato
165             _recordingPath = actualRecordingPath;
166             debugPrint('SpeechRecognitionService: Percorso file
167                 aggiornato a: $_recordingPath');
168         } else {
169             // Su altre piattaforme, usa AudioService
170             _recordingPath = await _audioService.stopRecording();
171             if (_recordingPath == null || _recordingPath!.isEmpty) {
172                 throw Exception('Errore nel fermare la registrazione');
173             }
174         }
175
176         // Verifica che il file esista prima di procedere
177         final audioFile = File(_recordingPath!);
178         if (!await audioFile.exists()) {
179             throw Exception('File audio non trovato dopo la
180                 registrazione: $_recordingPath');
181         }
182     }
183 }

```

```

179     final fileSize = await audioFile.length();
180     debugPrint('SpeechRecognitionService: File audio trovato,
        dimensione: $fileSize byte');
181
182     await _processRecording();
183   } catch (e) {
184     _emitError('Errore nello stop della registrazione: $e');
185     _updateState(RecognitionState.error);
186   }
187 }
188
189 /// Elabora la registrazione per il riconoscimento
190 Future<void> _processRecording() async {
191   if (_recordingPath == null || _currentTarget == null) {
192     _emitError('Dati di registrazione mancanti');
193     _updateState(RecognitionState.error);
194     return;
195   }
196
197   try {
198     /// Usa il servizio VOSK per il riconoscimento
199     final result = await _voskService.startRecognition(
        _currentTarget!, _recordingPath!);
200
201     /// Emetti il risultato
202     _resultController.add(result);
203     _updateState(RecognitionState.completed);
204
205     /// Elimina il file audio dopo l'elaborazione
206     await _deleteRecordingFile(_recordingPath!);
207   } catch (e) {
208     _emitError('Errore nel processamento dell\'audio: $e');
209
210     /// Crea un risultato fallback con similarita' bassa
211     final fallbackResult = RecognitionResult(
212       text: 'errore nel riconoscimento',
213       confidence: 0.1,
214       similarity: 0.0,
215       isCorrect: false,
216       duration: const Duration(seconds: 1),
217     );
218
219     _resultController.add(fallbackResult);
220     _updateState(RecognitionState.completed);

```

```

221
222     // Tenta comunque di eliminare il file
223     await _deleteRecordingFile(_recordingPath!);
224 }
225 }
226
227 /// Elimina il file di registrazione per risparmiare spazio
228 Future<void> _deleteRecordingFile(String filePath) async {
229     try {
230         final file = File(filePath);
231         if (await file.exists()) {
232             await file.delete();
233             debugPrint('SpeechRecognitionService: File di registrazione
                eliminato: $filePath');
234         }
235     } catch (e) {
236         debugPrint('SpeechRecognitionService: Errore nell\'
            eliminazione del file di registrazione: $e');
237         // Non solleviamo l'errore per non interrompere il flusso dell
            'app
238     }
239 }
240
241 /// Reimposta lo stato del servizio
242 Future<void> reset() async {
243     debugPrint('SpeechRecognitionService: Reset chiamato');
244
245     try {
246         // Ferma qualsiasi registrazione in corso
247         if (_currentState == RecognitionState.recording) {
248             try {
249                 if (Platform.isLinux && _linuxRecorder != null) {
250                     await _linuxRecorder!.stop();
251                 } else {
252                     await _audioService.stopRecording();
253                 }
254             } catch (e) {
255                 // Ignora gli errori qui
256                 debugPrint('SpeechRecognitionService: Errore fermando la
                    registrazione durante reset: $e');
257             }
258         }
259
260         // Resetta l'audio service

```

```

261     try {
262         await _audioService.reset();
263     } catch (e) {
264         // Ignora gli errori qui
265         debugPrint('SpeechRecognitionService: Errore nel reset dell
                \'audio service: $e');
266     }
267
268     _currentTarget = null;
269     _recordingPath = null;
270     _updateState(RecognitionState.idle);
271 } catch (e) {
272     _emitError('Errore nel reset: $e');
273     _updateState(RecognitionState.error);
274 }
275 }
276
277 /// Aggiorna lo stato e notifica gli ascoltatori
278 void _updateState(RecognitionState state) {
279     _currentState = state;
280     _stateController.add(state);
281     debugPrint('SpeechRecognitionService: Stato aggiornato a $state'
                );
282 }
283
284 /// Emette un errore sullo stream di errori
285 void _emitError(String message) {
286     debugPrint('SpeechRecognitionService: ERRORE - $message');
287     _errorController.add(message);
288 }
289
290 /// Rilascia le risorse del servizio
291 Future<void> dispose() async {
292     try {
293         if (_currentState == RecognitionState.recording) {
294             try {
295                 if (Platform.isLinux && _linuxRecorder != null) {
296                     await _linuxRecorder!.stop();
297                 } else {
298                     await _audioService.stopRecording();
299                 }
300             } catch (e) {
301                 // Ignora errori qui
302             }

```

```

303     }
304
305     if (_linuxRecorder != null) {
306       _linuxRecorder!.dispose();
307     }
308
309     await _audioService.dispose();
310
311     await _stateController.close();
312     await _resultController.close();
313     await _errorController.close();
314     await _volumeController.close();
315
316     _isInitialized = false;
317   } catch (e) {
318     debugPrint('SpeechRecognitionService: Errore nel dispose: $e')
319     ;
320   }
321 }

```

Listing 8.2: speech_recognition_service.dart

8.3 AudioService

E' il Servizio Principe, quello che si preoccupa di Registrare l'audio dal microfono. Per Linux è stato pensato un modulo aggiuntivo che sfrutta i vari plugin audio per la registrazione, in quanto, diversamente da macOS e Windows, può avere diverse tipologie di configurazione. Per comodità, all'interno del repository è stato inglobato **fmedia** con uno script di installer (fatto da me) per permettere la sua installazione in modo agevole e semplice.

```

1 // lib/services/audio_service.dart
2
3 import 'dart:async';
4 import 'dart:io';
5 import 'package:flutter/foundation.dart';

```

```

6 import 'package:path_provider/path_provider.dart';
7 import 'package:path/path.dart' as p;
8 import 'package:record/record.dart';
9 import 'package:record_platform_interface/record_platform_interface.
    dart';
10
11 // Importa il tuo enum AudioState da models/enums.dart
12 import '../models/enums.dart';
13
14 /// Servizio che gestisce la registrazione audio per OpenDSA:
    Reading.
15 /// Utilizza il plugin 'record' per fornire una registrazione audio
    di alta qualita'
16 /// ottimizzata per il riconoscimento vocale.
17 class AudioService {
18   // Singleton pattern
19   static final AudioService _instance = AudioService._internal();
20   factory AudioService() => _instance;
21
22   // Costanti di configurazione audio
23   static const int _defaultSampleRate = 16000; // 16 kHz
24   static const int _defaultBitRate = 16000; // 16 kbps
25   static const int _defaultNumChannels = 1; // Mono
26
27   // Costanti pubbliche per la gestione delle registrazioni
28   static const int _maxAttempts = 5;
29   static const Duration _delayBetweenRecordings = Duration(seconds:
    3);
30
31   // Getters pubblici per le costanti
32   int get maxAttempts => _maxAttempts;
33   Duration get delayBetweenRecordings => _delayBetweenRecordings;
34
35   // Stato interno
36   bool _isInitialized = false;
37   bool _isRecording = false;
38   int _currentAttempt = 0;
39   Timer? _volumeTimer;
40   late String _recordingPath;
41
42   // Stream controllers per gli eventi in tempo reale
43   final _volumeController = StreamController<double>.broadcast();
44   final _stateController = StreamController<AudioState>.broadcast();
45   final _progressController = StreamController<int>.broadcast();

```

```

46
47 // Istanza del recorder
48 final Record _audioRecorder = Record();
49
50 // Costruttore privato per il singleton
51 AudioService._internal();
52
53 // Getters pubblici per lo stato
54 Stream<double> get volumeLevel => _volumeController.stream;
55 Stream<AudioState> get audioState => _stateController.stream;
56 Stream<int> get recordingProgress => _progressController.stream;
57 bool get isInitialized => _isInitialized;
58 bool get isRecording => _isRecording;
59 int get currentAttempt => _currentAttempt;
60 bool get isSessionComplete => _currentAttempt >= maxAttempts;
61
62 /// Inizializza il servizio audio e verifica i permessi necessari
63 Future<void> initialize() async {
64     if (_isInitialized) return;
65
66     try {
67         debugPrint('AudioService: Inizializzazione...');
68
69         // Verifica i permessi di registrazione (solo per Android/iOS)
70         if (Platform.isAndroid || Platform.isIOS) {
71             final hasPermission = await _audioRecorder.hasPermission()
72                 .timeout(const Duration(seconds: 3), onTimeout: () {
73                     debugPrint('AudioService: Timeout nella verifica dei
74                         permessi');
75                     return false;
76                 });
77             if (!hasPermission) {
78                 throw Exception('Permessi audio non concessi');
79             }
80         } else {
81             // Su Linux, macOS e Windows, verifichiamo che il microfono
82             // sia attivo
83             debugPrint('AudioService: Piattaforma: ${Platform.
84                 operatingSystem}');
85             if (Platform.isLinux) {
86                 // Esegui un test veloce per verificare se il microfono e'
87                 // accessibile
88                 try {

```



```

86         Process.runSync('arecord', ['-l']);
87         debugPrint('AudioService: Test microfono con arecord
           riuscito');
88     } catch (e) {
89         debugPrint('AudioService: Test microfono fallito: $e');
90         // Non blocchiamo l'inizializzazione, ma logghiamo l'
           errore
91     }
92 }
93 }
94
95 // Prepara la directory per le registrazioni
96 final appDocDir = await getApplicationDocumentsDirectory()
97     .timeout(const Duration(seconds: 3), onTimeout: () {
98         debugPrint('AudioService: Timeout nell\'ottenimento della
           directory');
99         throw Exception('Timeout nell\'accesso al filesystem');
100     });
101
102 final recordingsDir = Directory('${appDocDir.path}/
           OpenDSA_recordings');
103 if (!await recordingsDir.exists()) {
104     await recordingsDir.create(recursive: true)
105         .timeout(const Duration(seconds: 3), onTimeout: () {
106             debugPrint('AudioService: Timeout nella creazione della
               directory');
107             throw Exception('Timeout nella creazione della directory');
108         });
109 }
110
111 // Aggiungiamo un timestamp per evitare conflitti di file
112 String timestamp = DateTime.now().millisecondsSinceEpoch.
           toString();
113 _recordingPath = '${recordingsDir.path}/recording_${timestamp}.
           wav';
114
115 debugPrint('AudioService: Path di registrazione impostato:
           $_recordingPath');
116
117 _isInitialized = true;
118 _updateState(AudioState.stopped);
119 debugPrint('AudioService: Inizializzazione completata');
120 } catch (e) {

```

```

121     debugPrint('AudioService: Errore nell\'inizializzazione: $e');
122     rethrow;
123   }
124 }
125
126 /// Avvia una nuova registrazione audio
127 Future<String> startRecording() async {
128   if (!_isInitialized) {
129     try {
130       await initialize().timeout(const Duration(seconds: 3),
131         onTimeout: () {
132           debugPrint('AudioService: Timeout nell\'inizializzazione')
133             ;
134           return;
135         });
136     } catch (e) {
137       debugPrint('AudioService: Errore nell\'inizializzazione: $e'
138         );
139       return '';
140     }
141   }
142
143   if (_isRecording) {
144     // Se stiamo gia' registrando, restituiamo semplicemente il
145     path
146     return _recordingPath;
147   }
148
149   try {
150     _currentAttempt++;
151     _progressController.add(_currentAttempt);
152
153     try {
154       // Verificare se la registrazione precedente e' ancora
155       attiva e fermarla
156       if (await _audioRecorder.isRecording()) {
157         await stopRecording();
158       }
159     } catch (e) {
160       // Ignora gli errori qui, potrebbe non esserci una
161       registrazione attiva
162       debugPrint('AudioService: Errore durante la verifica/stop
163         della registrazione precedente: $e');
164     }
165   }

```

```

158
159 // Configura e avvia la registrazione con timeout
160 await _audioRecorder.start(
161     encoder: AudioEncoder.wav,
162     bitRate: _defaultBitRate,
163     samplingRate: _defaultSampleRate,
164     numChannels: _defaultNumChannels,
165     path: _recordingPath,
166 ).timeout(const Duration(seconds: 3), onTimeout: () {
167     debugPrint('AudioService: Timeout nell\'avvio della
        registrazione');
168     return;
169 });
170
171 _isRecording = true;
172 _updateState(AudioState.recording);
173 _startVolumeMonitoring();
174
175 debugPrint('AudioService: Registrazione avviata (attempt
        $_currentAttempt)');
176 return _recordingPath;
177 } catch (e) {
178     debugPrint('AudioService: Errore nell\'avvio della
        registrazione: $e');
179     // Ritorniamo un path vuoto per segnalare il fallimento
180     return '';
181 }
182 }
183
184 /// Ferma la registrazione corrente
185 Future<String> stopRecording() async {
186     if (!_isRecording) {
187         // Non stiamo registrando
188         return '';
189     }
190
191     try {
192         _stopVolumeMonitoring();
193
194         // Gestione degli errori migliorata per le piattaforme Linux
195         try {
196             await _audioRecorder.stop().timeout(const Duration(seconds:
                3), onTimeout: () {
197                 debugPrint('AudioService: Timeout nello stop della

```

```

198         registrazione');
199         return null;
200     });
201 } catch (e) {
202     // Su Linux, potrebbe esserci un problema con fmedia, ma
203     // possiamo continuare
204     debugPrint('AudioService: Errore nella chiamata a stop, ma
205     continuiamo: $e');
206     // Creare un file vuoto se necessario per evitare errori
207     successivi
208     try {
209         final recordingFile = File(_recordingPath);
210         if (!await recordingFile.exists()) {
211             await recordingFile.writeAsBytes([]);
212         }
213     } catch (fileError) {
214         debugPrint('AudioService: Errore nella creazione del file
215         vuoto: $fileError');
216     }
217 }
218
219 _isRecording = false;
220 _updateState(AudioState.stopped);
221
222 debugPrint('AudioService: Registrazione fermata');
223 return _recordingPath;
224 } catch (e) {
225     debugPrint('AudioService: Errore nello stop della
226     registrazione: $e');
227     _isRecording = false;
228     _updateState(AudioState.stopped);
229     return '';
230 }
231 }
232
233 /// Monitora il volume durante la registrazione
234 void _startVolumeMonitoring() {
235     _volumeTimer?.cancel();
236     _volumeTimer = Timer.periodic(
237         const Duration(milliseconds: 100),
238         (timer) async {
239             if (!_isRecording) return;
240
241             try {

```

```

236         final amplitude = await _audioRecorder.getAmplitude();
237         // Il plugin 'record' fornisce un valore in dB, di solito
           tra -160 e 0
238         // Convertiamolo in un valore normalizzato tra 0.0 e 1.0
239         final volume = (amplitude.current + 160) / 160;
240         _volumeController.add(volume.clamp(0.0, 1.0));
241     } catch (err) {
242         // Ignora errori nel monitoraggio del volume
243         _volumeController.add(0.5); // Valore predefinito se c'e'
           un errore
244     }
245 },
246 );
247 }
248
249 void _stopVolumeMonitoring() {
250     _volumeTimer?.cancel();
251     _volumeTimer = null;
252 }
253
254 /// Aggiorna lo stato dell'audio e invia l'evento sullo stream
255 void _updateState(AudioState newState) {
256     _stateController.add(newState);
257 }
258
259 /// Reimposta lo stato del servizio
260 Future<void> reset() async {
261     try {
262         debugPrint('AudioService: Reset chiamato');
263         if (_isRecording) {
264             try {
265                 await stopRecording();
266             } catch (e) {
267                 debugPrint('AudioService: Errore nello stop della
           registrazione durante reset: $e');
268                 // Continuiamo anche in caso di errore nel fermare la
           registrazione
269                 _isRecording = false;
270                 _updateState(AudioState.stopped);
271             }
272         }
273         _currentAttempt = 0;
274         _progressController.add(_currentAttempt);
275     } catch (e) {

```

```

276     debugPrint('AudioService: Errore nel reset: $e');
277   }
278 }
279
280 /// Rilascia le risorse
281 Future<void> dispose() async {
282   if (_isRecording) {
283     try {
284       await stopRecording();
285     } catch (e) {
286       debugPrint('AudioService: Errore nello stop della
287         registrazione durante dispose: $e');
288     }
289   }
290   try {
291     await _audioRecorder.dispose();
292   } catch (e) {
293     debugPrint('AudioService: Errore nel dispose del recorder: $e'
294       );
295   }
296   await _volumeController.close();
297   await _stateController.close();
298   await _progressController.close();
299
300   _isInitialized = false;
301   debugPrint('AudioService: Risorse rilasciate');
302 }
303 }

```

Listing 8.3: audio_service.dart

Bibliografia

- [1] Alpha Cephei. *VOSK Documentation*. 2024. URL: <https://alphacephei.com/vosk/> (visitato il 15/02/2024).
- [2] Alpha Cephei. *VOSK Speech Recognition Toolkit*. 2024. URL: <https://github.com/alphacep/vosk-api> (visitato il 15/02/2024).
- [3] American Psychiatric Association. *Diagnostic and Statistical Manual of Mental Disorders*. 5^a ed. American Psychiatric Publishing, 2013. ISBN: 978-0890425558.
- [4] Wikipedia contributors. *Distanza di Levenshtein* — *Wikipedia, L'enciclopedia libera*. https://it.wikipedia.org/wiki/Distanza_di_Levenshtein. Accesso: 27 Febbraio 2025.
- [5] Cesare Cornoldi e Barbara Carretti. *DSA e disturbo del linguaggio: Evoluzione, assessment, intervento*. Bologna, Italia: Il Mulino, 2016. ISBN: 978-8815267337.
- [6] Cesare Cornoldi e Barbara Carretti. *Reading and Spelling Disorders in Different Orthographic Systems*. Routledge, 2016. ISBN: 978-1138937772.
- [7] Dart Language. *Dart 2.0 Language Specification*. 2021. URL: <https://dart.dev/guides> (visitato il 15/02/2024).
- [8] DeepMind. *DeepMind Research*. 2024. URL: <https://deepmind.com> (visitato il 15/02/2024).

- [9] *Direttiva (UE) 2016/2102 del Parlamento europeo e del Consiglio*. 26 Ott. 2016.
- [10] *Disposizioni per favorire l'accesso dei soggetti disabili agli strumenti informatici*. 9 Gen. 2004.
- [11] Brock L. Eide e Fernette F. Eide. *The Dyslexic Advantage: Unlocking the Hidden Potential of the Dyslexic Brain*. Hay House, 2011. ISBN: 978-1848509528.
- [12] Flutter Documentation. *Hot Reload Overview*. 2023. URL: <https://docs.flutter.dev/development/tools/hot-reload> (visitato il 15/02/2024).
- [13] Ian Goodfellow et al. “Generative Adversarial Nets”. In: *Advances in Neural Information Processing Systems*. Vol. 27. 2014, pp. 2672–2680.
- [14] Google. *Dart Developer Summit 2015: Sky Demo*. 2015. URL: <https://www.youtube.com/watch?v=PnIWl33YMwA> (visitato il 15/02/2024).
- [15] Google. *Flutter Technical Overview*. 2018. URL: <https://flutter.dev/docs/resources/technical-overview> (visitato il 15/02/2024).
- [16] Davide Masiero e Daniela Lucangeli. *DSA: Diagnosi e intervento nella pratica clinica e educativa*. Trento, Italia: Erickson, 2017. ISBN: 978-8859013839.
- [17] John McCarthy et al. “A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence”. In: *AI Magazine* 27.4 (1955), pp. 12–14.
- [18] Melanie Mitchell. *Artificial Intelligence: A Guide for Thinking Humans*. Farrar, Straus e Giroux, 2019. ISBN: 978-0374257835.
- [19] MIUR. *Alunni con Disturbi Specifici di Apprendimento (DSA)*. Report Statistico. Roma, Italia: Ministero dell'Istruzione, 2024.

- [20] *Nuove norme in materia di disturbi specifici di apprendimento in ambito scolastico*. 8 Ott. 2010.
- [21] OpenAI. *OpenAI Research*. 2024. URL: <https://openai.com> (visitato il 15/02/2024).
- [22] Daniel Povey et al. “The Kaldi Speech Recognition Toolkit”. In: *IEEE Workshop on Automatic Speech Recognition and Understanding*. Hawaii, US, 2011.
- [23] pub.dev. *Official Flutter Packages*. 2023. URL: <https://pub.dev/> (visitato il 15/02/2024).
- [24] Stuart Russell e Peter Norvig. *Artificial Intelligence: A Modern Approach*. 4^a ed. Pearson, 2021. ISBN: 978-0134610993.
- [25] *Section 508 of the Rehabilitation Act*. 1998. URL: <https://www.section508.gov/>.
- [26] Sally E. Shaywitz. *Overcoming Dyslexia: A New and Complete Science-Based Program for Overcoming Reading Problems at Any Level*. Vintage Books, 2003. ISBN: 978-0679781592.
- [27] Alan M. Turing. “Computing Machinery and Intelligence”. In: *Mind* 59.236 (1950), pp. 433–460. DOI: [10.1093/mind/LIX.236.433](https://doi.org/10.1093/mind/LIX.236.433).
- [28] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017, pp. 5998–6008.
- [29] W3C. *Web Accessibility Initiative (WAI)*. 2024. URL: <https://www.w3.org/WAI/> (visitato il 15/02/2024).
- [30] W3C. *Web Content Accessibility Guidelines (WCAG) 2.0*. Technical Standard. W3C, 2008. URL: <https://www.w3.org/TR/WCAG20/>.

- [31] W3C. *Web Content Accessibility Guidelines (WCAG) 2.1*. Technical Standard. W3C, 2018. URL: <https://www.w3.org/TR/WCAG21/>.
- [32] Joseph Weizenbaum. “ELIZA — A Computer Program for the Study of Natural Language Communication Between Man and Machine”. In: *Communications of the ACM* 9.1 (1966), pp. 36–45. DOI: [10.1145/365153.365168](https://doi.org/10.1145/365153.365168).
- [33] World Health Organization. *International Classification of Functioning, Disability and Health*. Classification. Geneva, Switzerland: WHO, 2001.